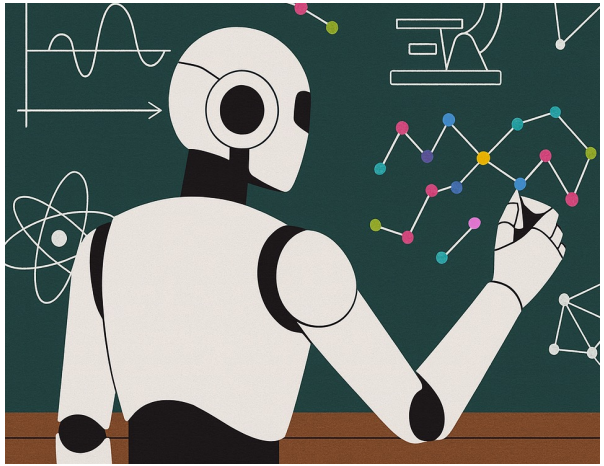
If you have any questions, feel free to reach out!
xuemei.gu@uni-jena.de

AI for Quantum Physics



Winter semester 2025/26

Today's topic:

- Diffusion models
- Surrogate models /deep dreaming

Deep Generative Learning

Learning to generate data

Samples from a Data Distribution

Train

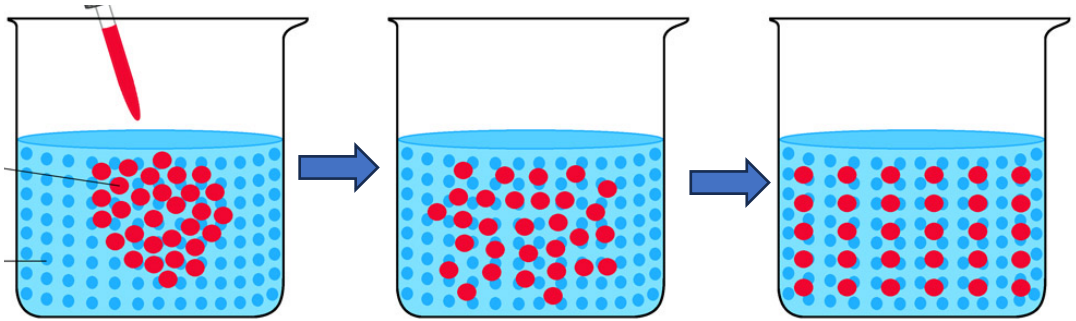
Neural Network

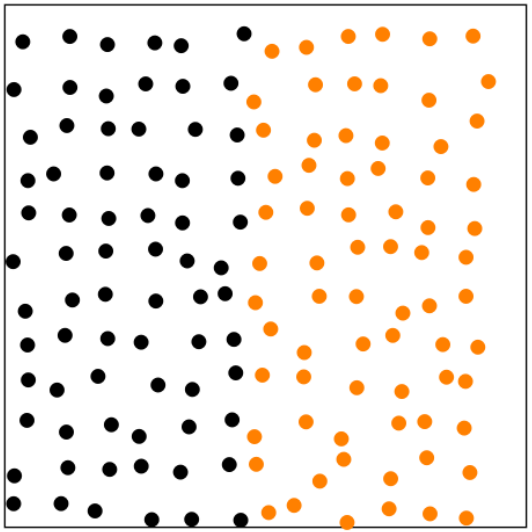
Sample

Diffusion Process

- Diffusion models are inspired by non-equilibrium thermodynamics.

- For a small fraction of the time, it is difficult to determine whether particles are moving in the direction of mixing or in the opposite direction.

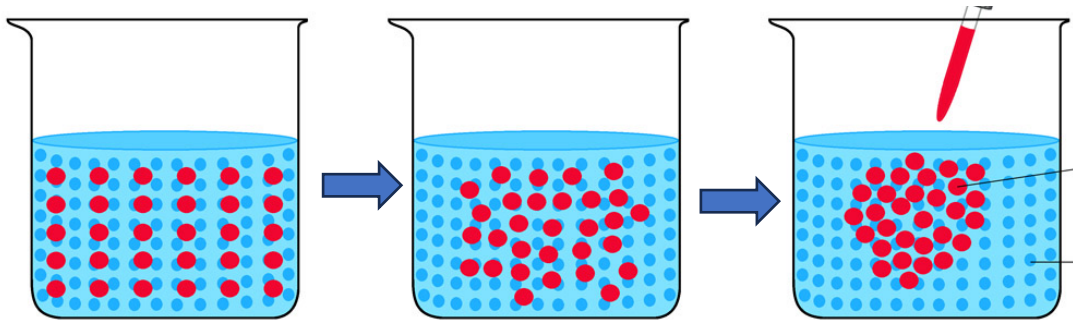




Diffusion Process

- If we look at the movement of a single molecule on a very short time scale, it follows a Gaussian distribution.

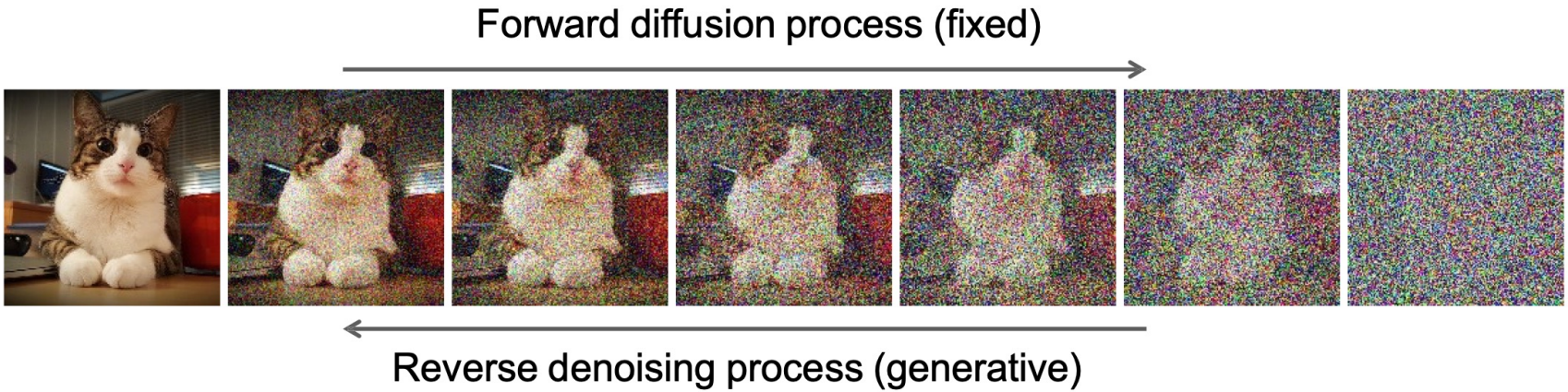
- Since the direction of mixing and the opposite direction are the same in a very short time, the opposite direction also follows a Gaussian distribution.



Denoising Diffusion Models

Denoising diffusion models consist of two processes:

- Forward diffusion process that gradually adds noise to input
- Reverse denoising process that learns to generate data by denoising



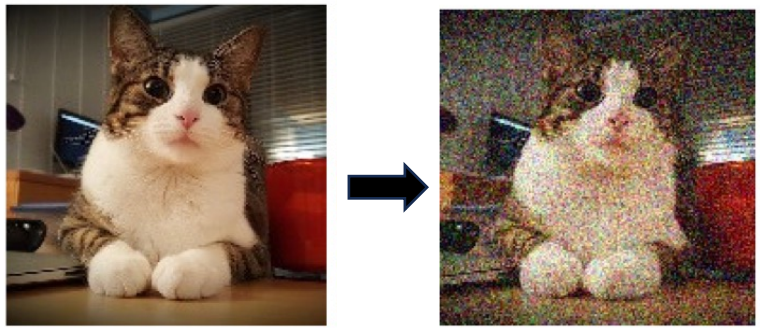
Denoising Diffusion Models

Forward Diffusion Process Details:

We take an image from the training data, denoted by x_0, and blend it with Gaussian noise independently for each pixel to give a noise corrupted image x_1 defined by

$$x_1 = \sqrt{1-\beta_1}x_0 + \sqrt{\beta_1}\epsilon$$

Gaussian noise $\mathcal{N}(0, I)$



x_0 x_1

An alternative representation is

$$q(x_1|x_0) = N(x_1; \sqrt{1-\beta_1}x_0, \beta_1 I)$$

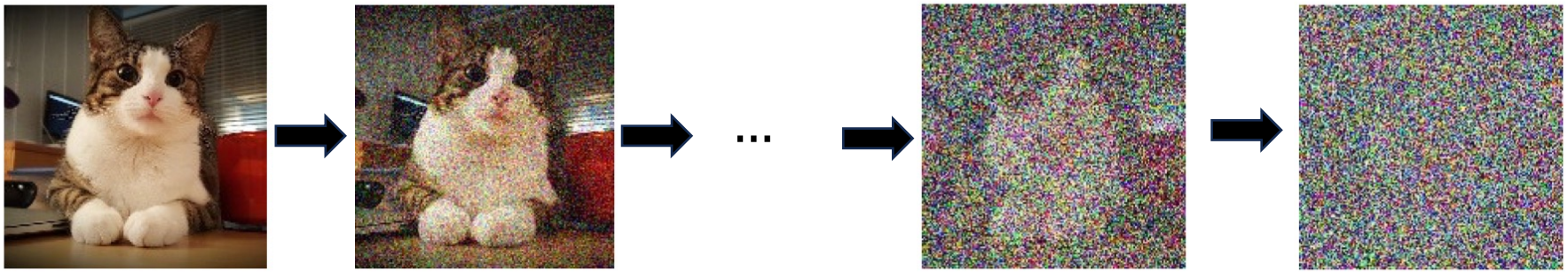
Denoising Diffusion Models

Forward Diffusion Process Details:

We repeat the process with additional independent Gaussian noise steps to give a sequence of increasingly noisy images. Each successive image is given by

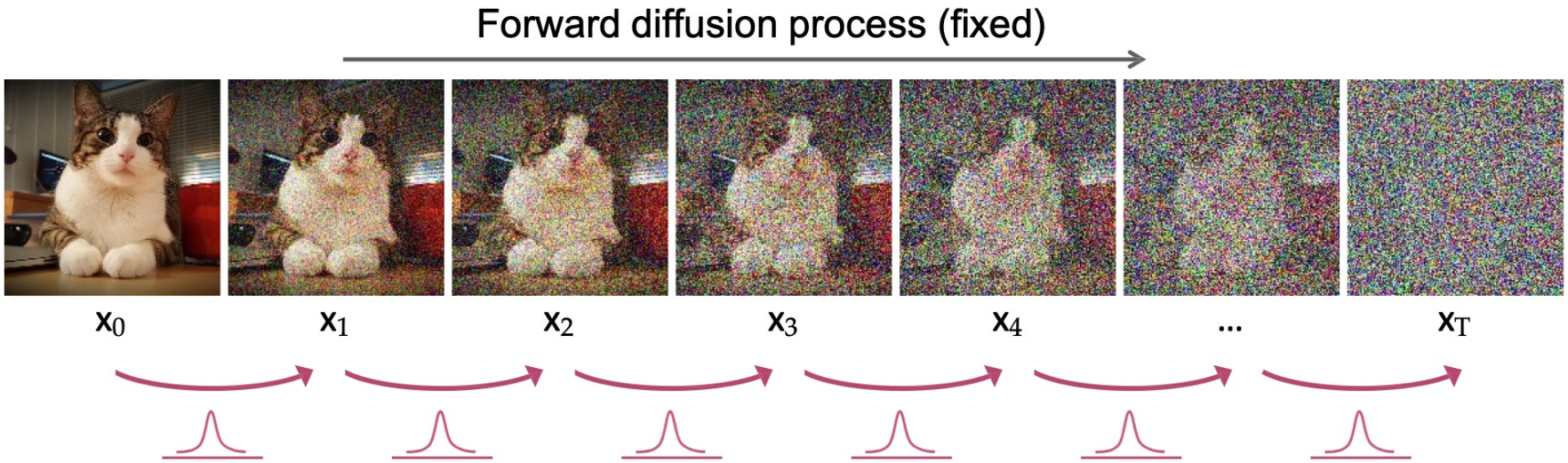
$$x_t = \sqrt{1-\beta_t}x_{t-1} + \sqrt{\beta_t}\epsilon$$

$$q(x_t|x_{t-1}) = N(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I)$$



x_0 x_1 x_{t-1} x_t

Denoising Diffusion Models



the **known** forward process

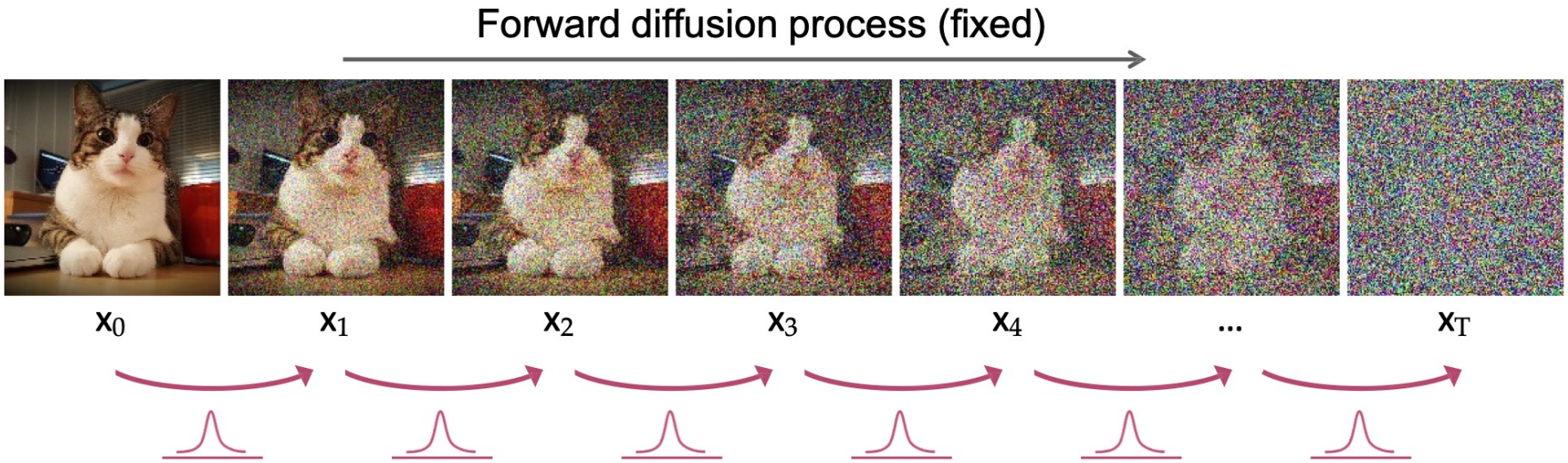
$$q(x_{1:T} \mid x_0) = \prod_{t=1}^{T} q(x_t \mid x_{t-1})$$

Probability Chain Rule (Markov Chain)

$$q(x_t|x_{t-1}) = N(x_t; \sqrt{1-\beta_t}x_{t-1}, \beta_t I)$$

Conditional Gaussian

Denoising Diffusion Models

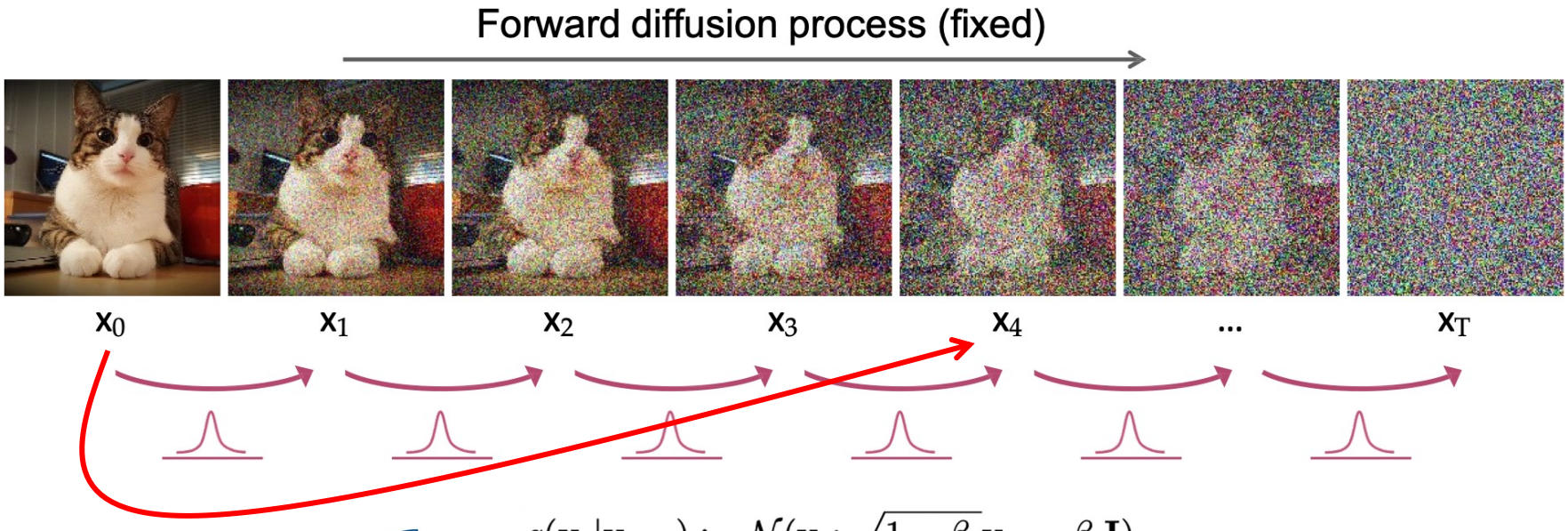


$q(x_t|x_{t-1}) = \mathcal{N}(x_t; (\sqrt{1-\beta_t}\ )x_{t-1}, \beta_t I)$ Conditional Gaussian

β_t is the *variance schedule* at the diffusion step t

Example: $0 < \beta_1 < \beta_2 < \cdots < \beta_T < 1$, typical value range $[0.0001, 0.02]$, with $T = 1000$

Denoising Diffusion Models



Markov Property

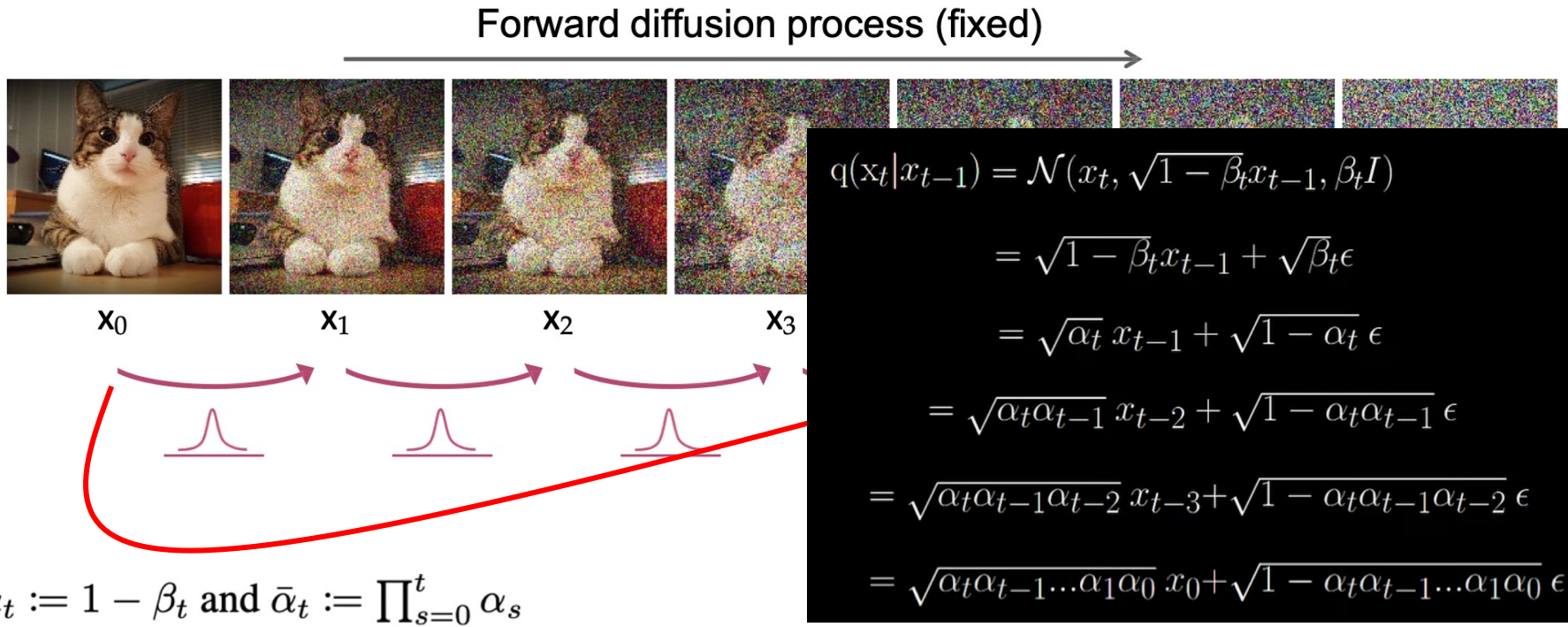
$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) := \mathcal{N}(\mathbf{x}_t; \sqrt{1-\beta_t}\mathbf{x}_{t-1}, \beta_t\mathbf{I})$$

$$q(x_t|x_0) = \mathcal{N}(x_t; \sqrt{\bar{\alpha}_t}x_0, (1-\bar{\alpha}_t)\mathbf{I})$$ ← Diffusion Kernel

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t}\,\mathbf{x}_0 + \sqrt{(1-\bar{\alpha}_t)}\,\epsilon \quad \text{where} \quad \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

$$\alpha_t := 1 - \beta_t \text{ and } \bar{\alpha}_t := \prod_{s=0}^{t} \alpha_s$$

Denoising Diffusion Models



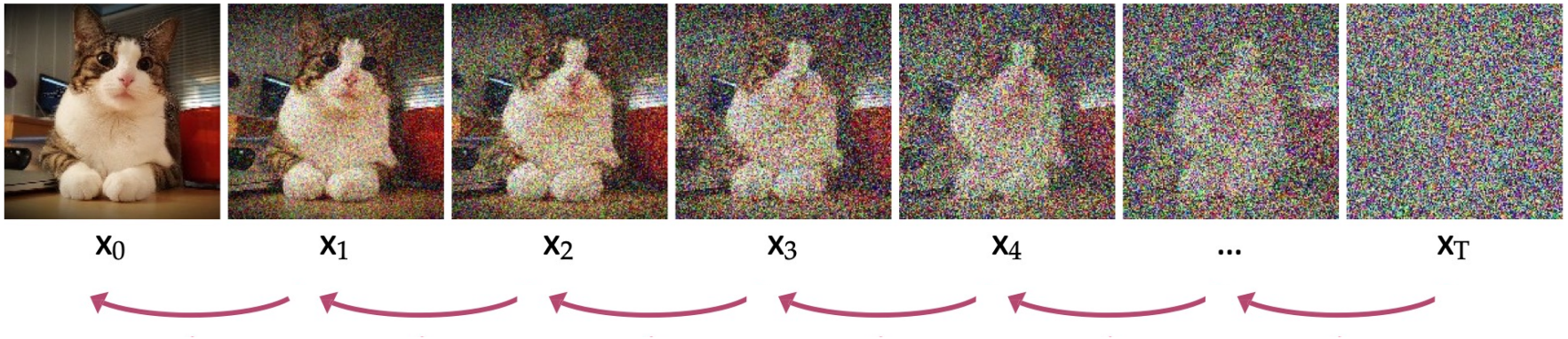
$\alpha_t := 1 - \beta_t$ and $\bar{\alpha}_t := \prod_{s=0}^{t} \alpha_s$

$$q(x_t \mid x_0) = \mathcal{N}\big(x_t; \sqrt{\bar{\alpha}_t}\, x_0,\ (1-\bar{\alpha}_t)\mathbf{I}\big)$$

samples from an arbitrary forward step are also Gaussian-distributed!

Denoising Diffusion Models

Reverse Diffusion Process Details:



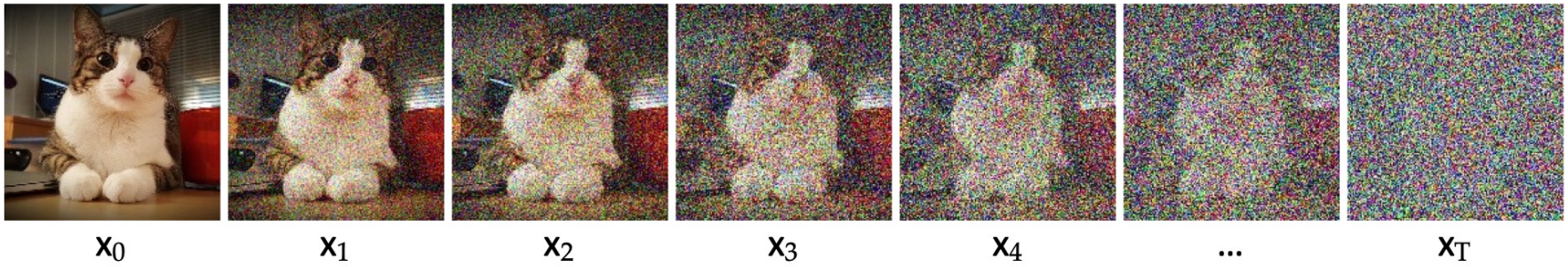
To reverse the process we would need to know the transition probabilities $q(x_{t-1} \mid x_t)$. They cannot be directly calculated. Instead we learn a model p_θ to approximate these conditional probabilities.

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1}|x_t)$$ Probability Chain Rule (Markov Chain)

$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$$ Conditional Gaussian

Denoising Diffusion Models

Reverse Diffusion Process Details:



$$p_\theta(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_\theta(x_t, t), \Sigma_\theta(x_t, t))$$

This does not have a closed form, but Ho, Jain and Abbeel (2020) derived the following approximation

$$\tilde{\mu}_\theta(x_t, t) \approx \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \epsilon_t \right)$$

We do not know $\boldsymbol{\epsilon_t}$, but we can train a neural network $\epsilon_\theta(\boldsymbol{x_t}, t)$ to approximate it:

$$\text{Minimize } L(\theta) = \left|\left|\boldsymbol{\epsilon_t} - \epsilon_\theta(\boldsymbol{x_t}, t)\right|\right|_2^2$$

How do we train Denoising Diffusion Models

What is the loss function? (Ho et al. NeurIPS 2020)

$$L_{\text{simple}} = \mathbb{E}_{\mathbf{x}_0 \sim q(\mathbf{x}_0), \epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), t \sim \mathcal{U}(1,T)} \left[||\epsilon - \epsilon_\theta(\underbrace{\sqrt{\bar{\alpha}_t}\, \mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\, \epsilon}_{\mathbf{x}_t}, t)||^2 \right]$$

Algorithm 1 Training

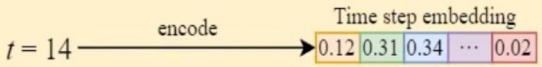
1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \text{Uniform}(\{1, \dots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$
6: **until** converged

U-Net autoencoder takes x(t) as input and simply predict a noise. The goal of the training is to generate a noise pattern that is unit normal.

How do we train Denoising Diffusion Models

For each training step:

1. Randomly select a time step & encode it



2. Add noise to image

$$x_t = \sqrt{\bar{a}_t}\, x_0 + \sqrt{1 - \bar{a}_t}\, \varepsilon$$

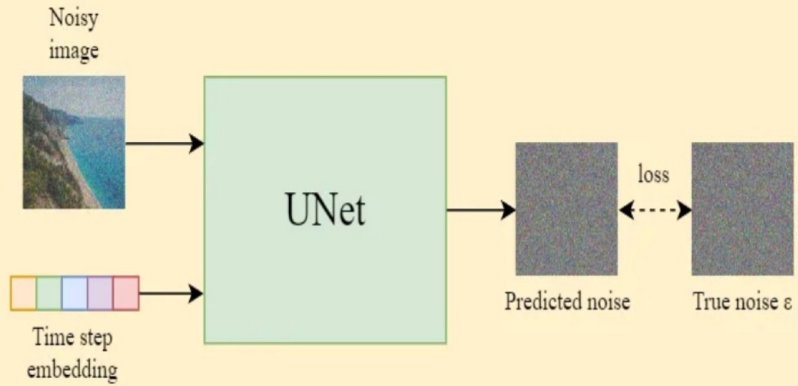
Adjust the amount of noise according to the time step t

$$\varepsilon \sim \mathcal{N}(0, 1)$$

$$\alpha_t = 1 - \beta_t$$

$$\bar{\alpha}_t = \prod_{i=1}^{t} \alpha_i$$

3. Train the UNet



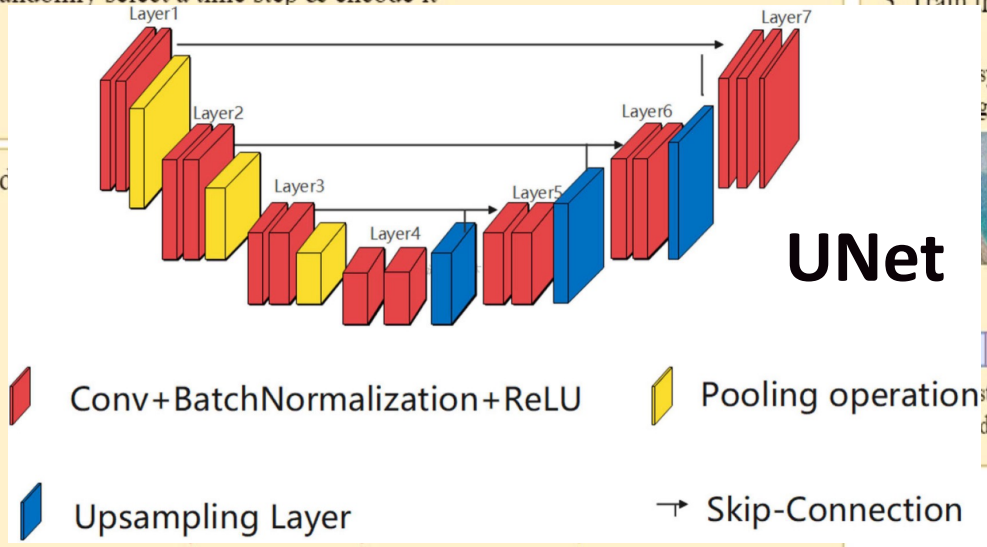
From Steins (medium.com)

How do we train Denoising Diffusion Models

For each training step:

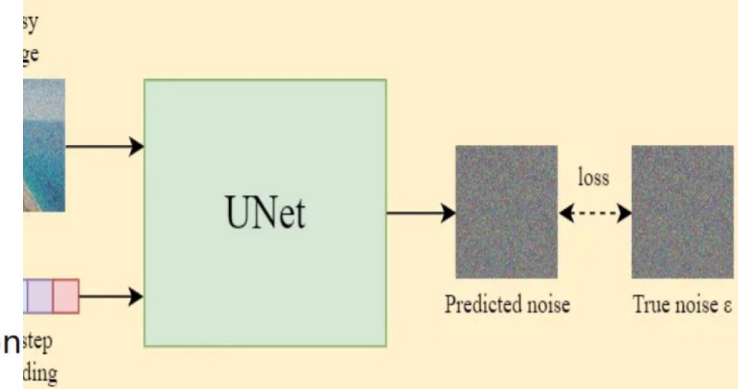
1. Randomly select a time step & encode it

2. Ad



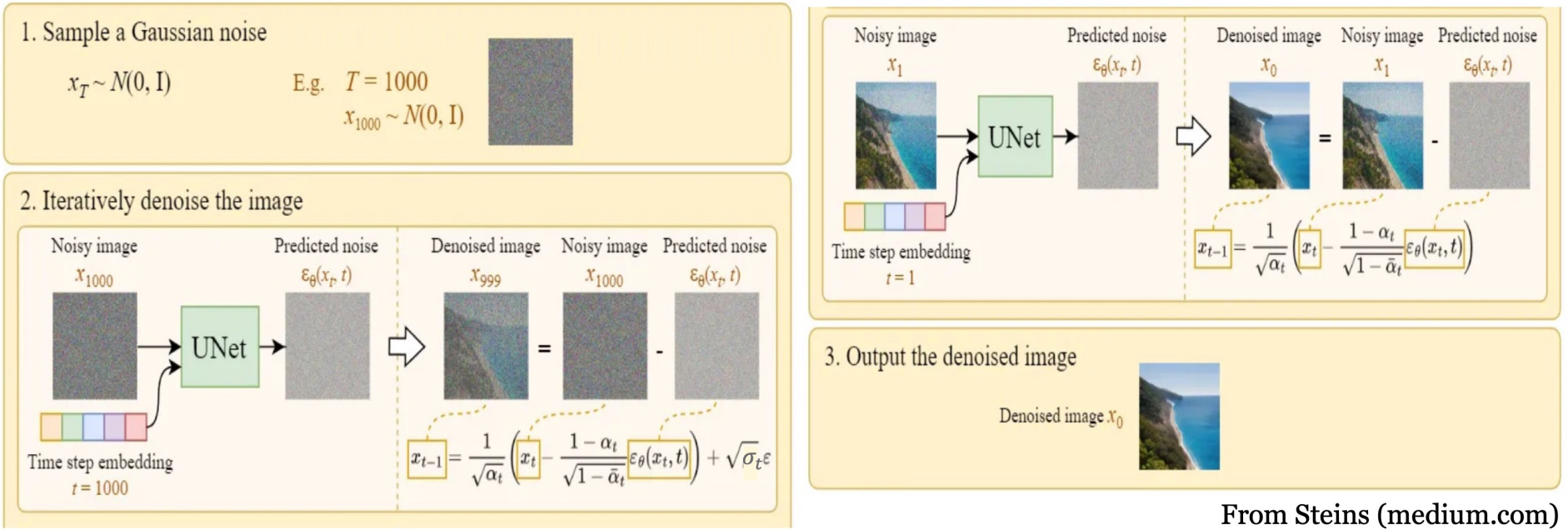
Adjust the amount of noise according to the time step t

3. Train the UNet



From Steins (medium.com)

Denoising Diffusion Models



From Steins (medium.com)

Denoising Diffusion Models

Training and Sample Generation

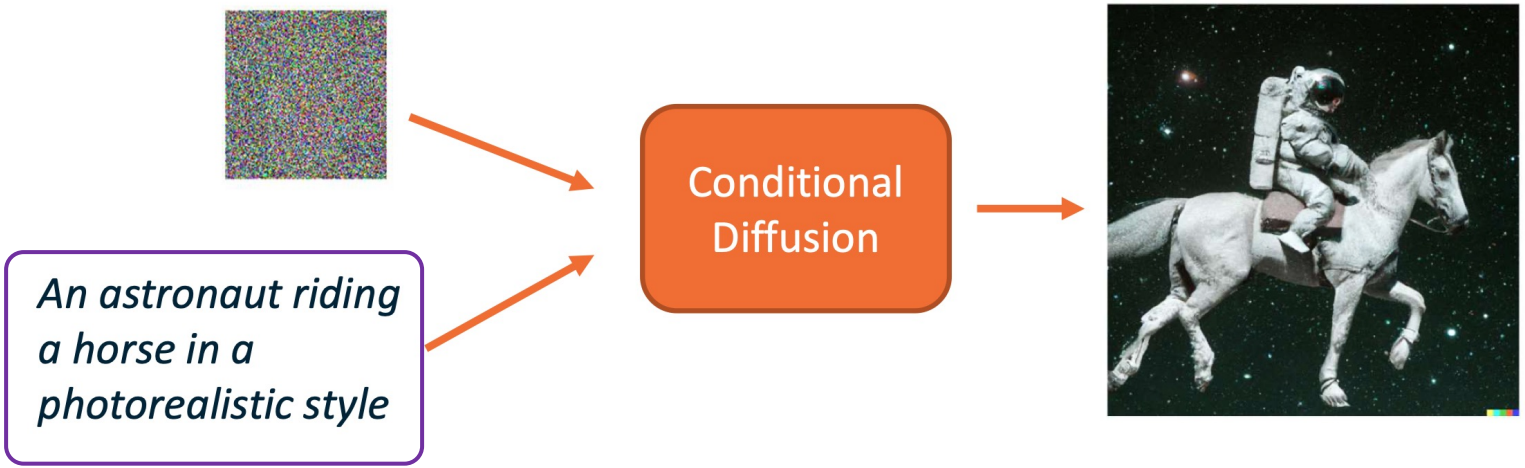
Algorithm 1 Training

1: **repeat**
2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$
3: $t \sim \text{Uniform}(\{1, \dots, T\})$
4: $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
5: Take gradient descent step on
$$\nabla_\theta \left\| \boldsymbol{\epsilon} - \boldsymbol{\epsilon}_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1-\bar{\alpha}_t}\boldsymbol{\epsilon}, t) \right\|^2$$
6: **until** converged

Algorithm 2 Sampling

1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$
2: **for** $t = T, \dots, 1$ **do**
3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$
4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\bar{\alpha}_t}} \boldsymbol{\epsilon}_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$
5: **end for**
6: **return** $\mathbf{x}_0$

Conditional Diffusion Models

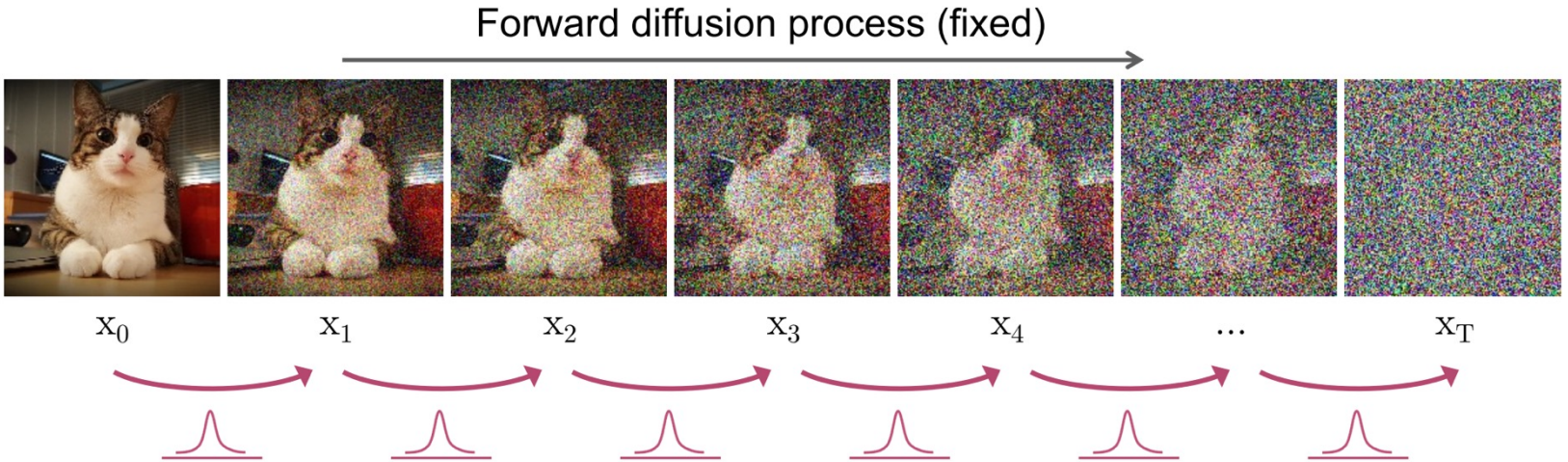


Simple idea: just condition the model on some text labels y

$$\epsilon_\theta(x_t, y, t)$$

Score-based Generative Modeling with Differential Equations

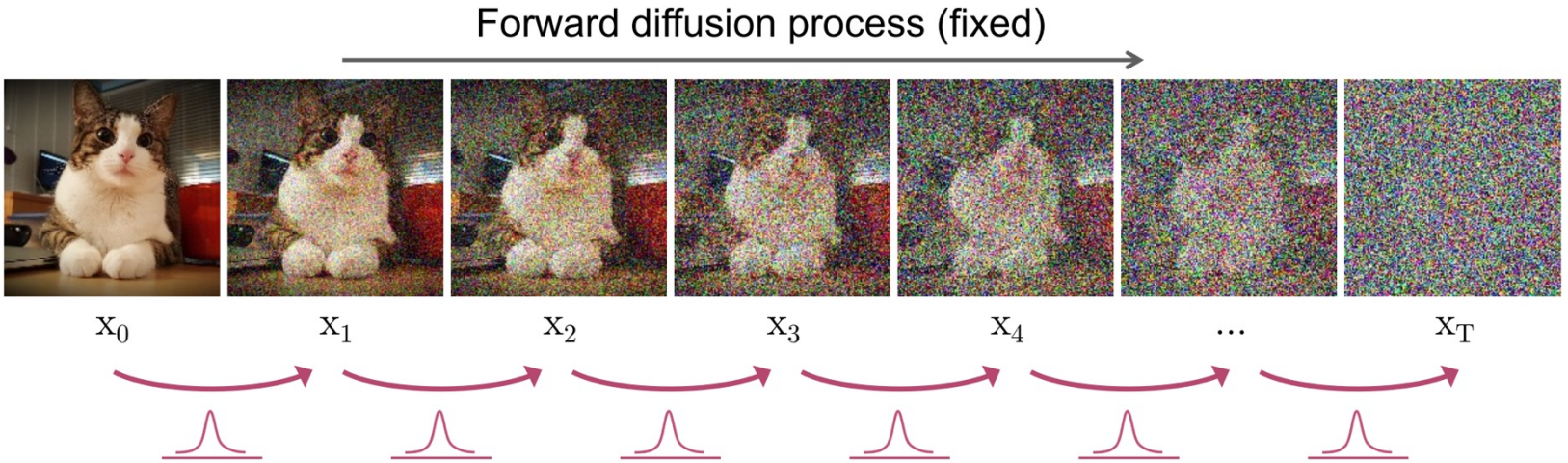
https://www.youtube.com/watch?v=cS6JQpEY9cs



$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1-\beta_t}\,\mathbf{x}_{t-1}, \beta_t\mathbf{I})$$

$$\mathbf{x}_t = \sqrt{1-\beta_t}\,\mathbf{x}_{t-1} + \sqrt{\beta_t}\,\mathcal{N}(\mathbf{0}, \mathbf{I})$$

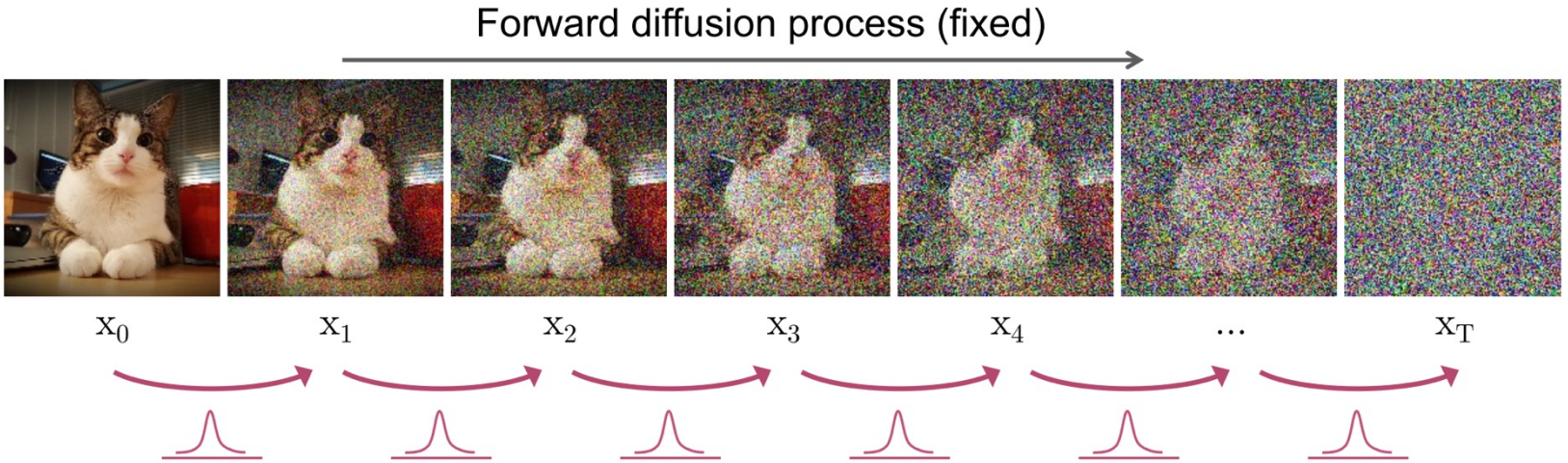
Score-based Generative Modeling with Differential Equations



$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1-\beta_t}\,\mathbf{x}_{t-1}, \beta_t\mathbf{I})$$

$$\begin{aligned}\mathbf{x}_t &= \sqrt{1-\beta_t}\,\mathbf{x}_{t-1} + \sqrt{\beta_t}\,\mathcal{N}(\mathbf{0},\mathbf{I}) \\ &= \sqrt{1-\beta(t)\Delta t}\,\mathbf{x}_{t-1} + \sqrt{\beta(t)\Delta t}\,\mathcal{N}(\mathbf{0},\mathbf{I}) \qquad (\beta_t := \beta(t)\Delta t)\end{aligned}$$

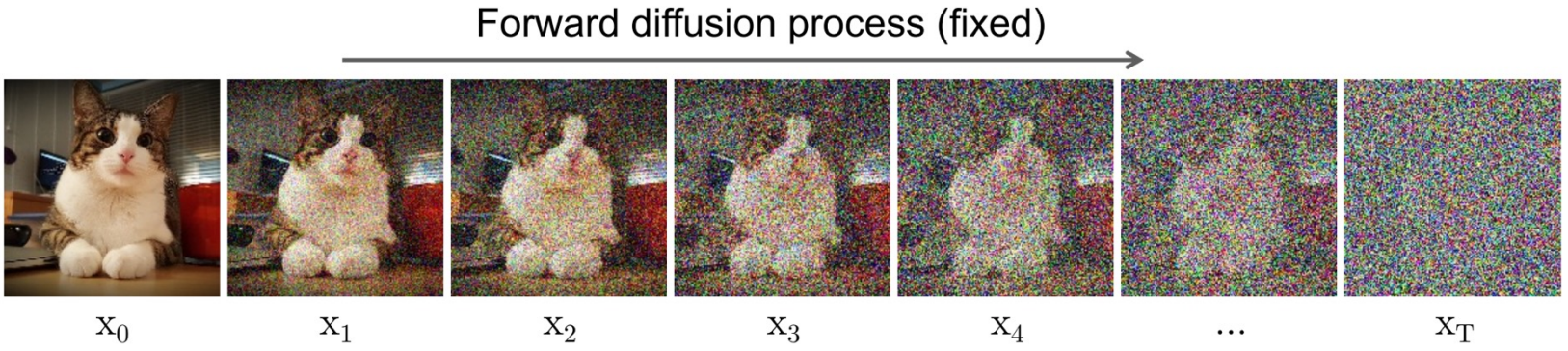
Score-based Generative Modeling with Differential Equations



$$q(\mathbf{x}_t|\mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{1-\beta_t}\,\mathbf{x}_{t-1}, \beta_t\mathbf{I})$$

$$\begin{aligned}
\mathbf{x}_t &= \sqrt{1-\beta_t}\,\mathbf{x}_{t-1} + \sqrt{\beta_t}\,\mathcal{N}(\mathbf{0},\mathbf{I}) \\
&= \sqrt{1-\beta(t)\Delta t}\,\mathbf{x}_{t-1} + \sqrt{\beta(t)\Delta t}\,\mathcal{N}(\mathbf{0},\mathbf{I}) && (\beta_t := \beta(t)\Delta t) \\
&\approx \mathbf{x}_{t-1} - \frac{\beta(t)\Delta t}{2}\mathbf{x}_{t-1} + \sqrt{\beta(t)\Delta t}\,\mathcal{N}(\mathbf{0},\mathbf{I}) && \text{(Taylor expansion)}
\end{aligned}$$

Score-based Generative Modeling with Differential Equations



$$\mathbf{x}_t \approx \mathbf{x}_{t-1} - \frac{\beta(t)\Delta t}{2}\mathbf{x}_{t-1} + \sqrt{\beta(t)\Delta t}\,\mathcal{N}(\mathbf{0}, \mathbf{I})$$

$$\mathrm{d}\mathbf{x}_t = -\frac{1}{2}\beta(t)\mathbf{x}_t\,\mathrm{d}t + \sqrt{\beta(t)}\,\mathrm{d}\boldsymbol{\omega}_t$$

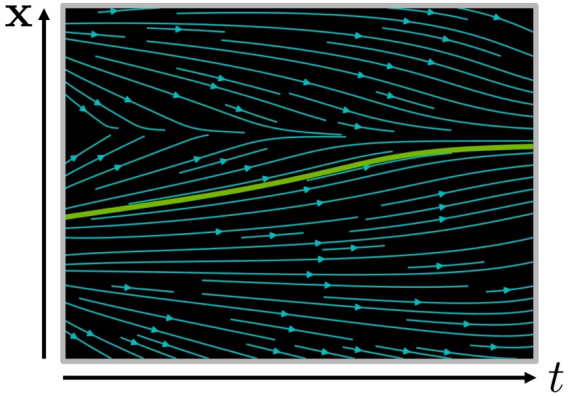
Stochastic Differential Equation (SDE)
describing the diffusion in infinitesimal limit

Song et al., "Score-Based Generative Modeling through Stochastic Differential Equations", ICLR, 2021

Differential Equations

Ordinary Differential Equation (ODE):

$$\frac{\mathrm{d}\mathbf{x}}{\mathrm{d}t} = \mathbf{f}(\mathbf{x}, t) \quad \text{or} \quad \mathrm{d}\mathbf{x} = \mathbf{f}(\mathbf{x}, t)\mathrm{d}t$$



Analytical Solution:

$$\mathbf{x}(t) = \mathbf{x}(0) + \int_0^t \mathbf{f}(\mathbf{x}, \tau)\mathrm{d}\tau$$

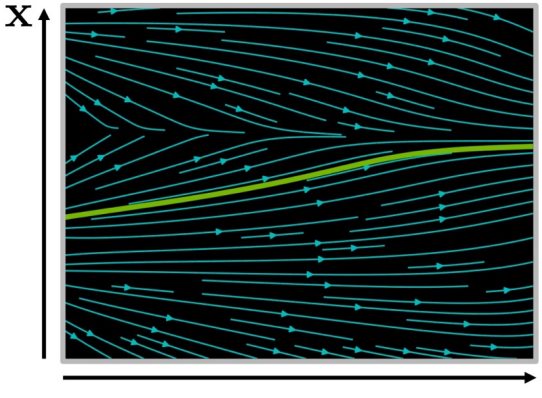
Iterative Numerical Solution:

$$\mathbf{x}(t + \Delta t) \approx \mathbf{x}(t) + \mathbf{f}(\mathbf{x}(t), t)\Delta t$$

Differential Equations

Ordinary Differential Equation (ODE):

$$\frac{\mathrm{d}\mathbf{x}}{\mathrm{d}t} = \mathbf{f}(\mathbf{x}, t) \quad \text{or} \quad \mathrm{d}\mathbf{x} = \mathbf{f}(\mathbf{x}, t)\mathrm{d}t$$



Analytical Solution:

$$\mathbf{x}(t) = \mathbf{x}(0) + \int_0^t \mathbf{f}(\mathbf{x}, \tau)\mathrm{d}\tau$$

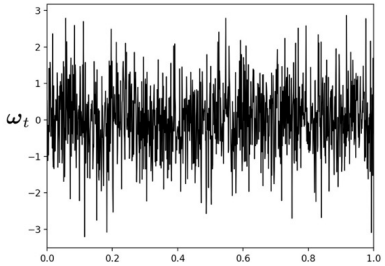
Iterative Numerical Solution:

$$\mathbf{x}(t + \Delta t) \approx \mathbf{x}(t) + \mathbf{f}(\mathbf{x}(t), t)\Delta t$$

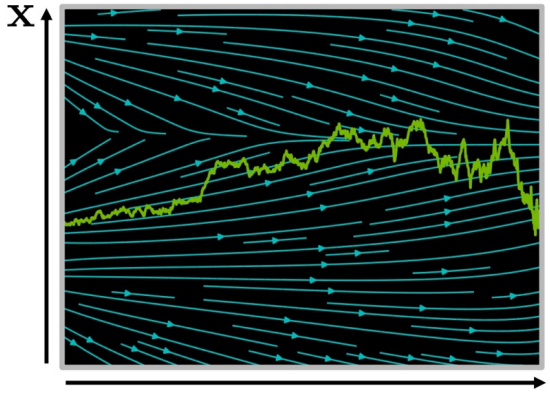
Stochastic Differential Equation (SDE):

$$\frac{\mathrm{d}\mathbf{x}}{\mathrm{d}t} = \underbrace{\mathbf{f}(\mathbf{x}, t)}_{\text{drift coefficient}} + \underbrace{\sigma(\mathbf{x}, t)\boldsymbol{\omega}_t}_{\text{diffusion coefficient}}$$

Wiener Process (Gaussian White Noise)



$$\left(\mathrm{d}\mathbf{x} = \mathbf{f}(\mathbf{x}, t)\mathrm{d}t + \sigma(\mathbf{x}, t)\mathrm{d}\boldsymbol{\omega}_t \right)$$

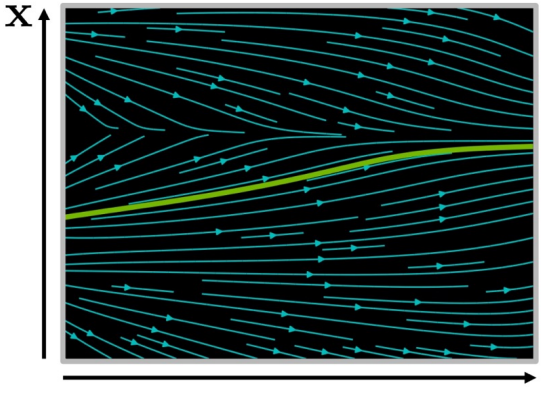


$$\mathbf{x}(t + \Delta t) \approx \mathbf{x}(t) + \mathbf{f}(\mathbf{x}(t), t)\Delta t + \sigma(\mathbf{x}(t), t)\sqrt{\Delta t}\,\mathcal{N}(\mathbf{0}, \mathbf{I})$$

Differential Equations

Ordinary Differential Equation (ODE):

$$\frac{\mathrm{d}\mathbf{x}}{\mathrm{d}t} = \mathbf{f}(\mathbf{x}, t) \quad \text{or} \quad \mathrm{d}\mathbf{x} = \mathbf{f}(\mathbf{x}, t)\mathrm{d}t$$



Analytical Solution:

$$\mathbf{x}(t) = \mathbf{x}(0) + \int_0^t \mathbf{f}(\mathbf{x}, \tau)\mathrm{d}\tau$$

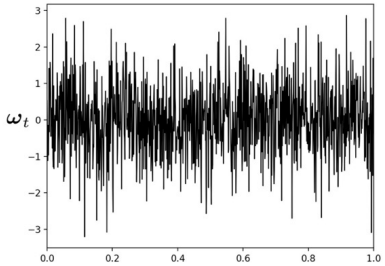
Iterative Numerical Solution:

$$\mathbf{x}(t + \Delta t) \approx \mathbf{x}(t) + \mathbf{f}(\mathbf{x}(t), t)\Delta t$$

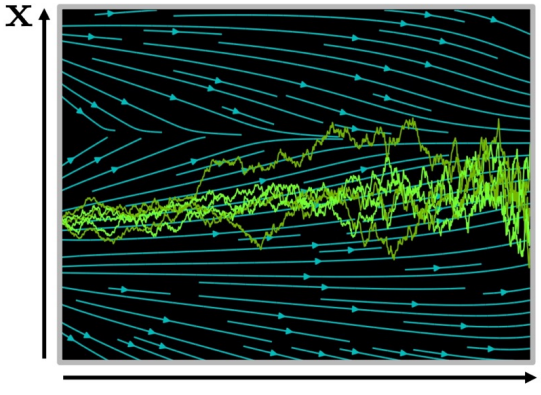
Stochastic Differential Equation (SDE):

$$\frac{\mathrm{d}\mathbf{x}}{\mathrm{d}t} = \underbrace{\mathbf{f}(\mathbf{x}, t)}_{\text{drift coefficient}} + \underbrace{\sigma(\mathbf{x}, t)\boldsymbol{\omega}_t}_{\text{diffusion coefficient}}$$

Wiener Process (Gaussian White Noise)

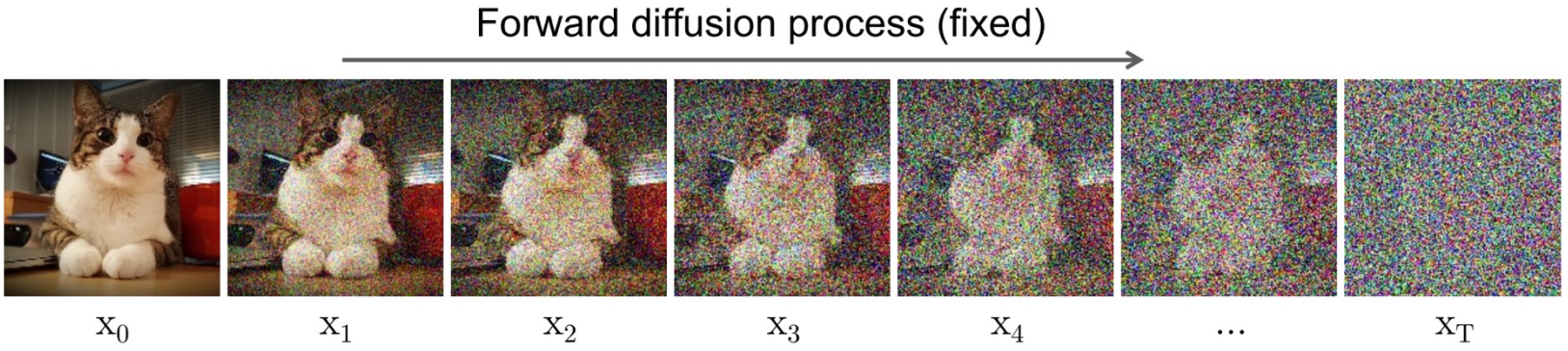


$$\left(\mathrm{d}\mathbf{x} = \mathbf{f}(\mathbf{x}, t)\mathrm{d}t + \sigma(\mathbf{x}, t)\mathrm{d}\boldsymbol{\omega}_t \right)$$



$$\mathbf{x}(t + \Delta t) \approx \mathbf{x}(t) + \mathbf{f}(\mathbf{x}(t), t)\Delta t + \sigma(\mathbf{x}(t), t)\sqrt{\Delta t}\,\mathcal{N}(\mathbf{0}, \mathbf{I})$$

Forward Diffusion Process as Stochastic Differential Equation



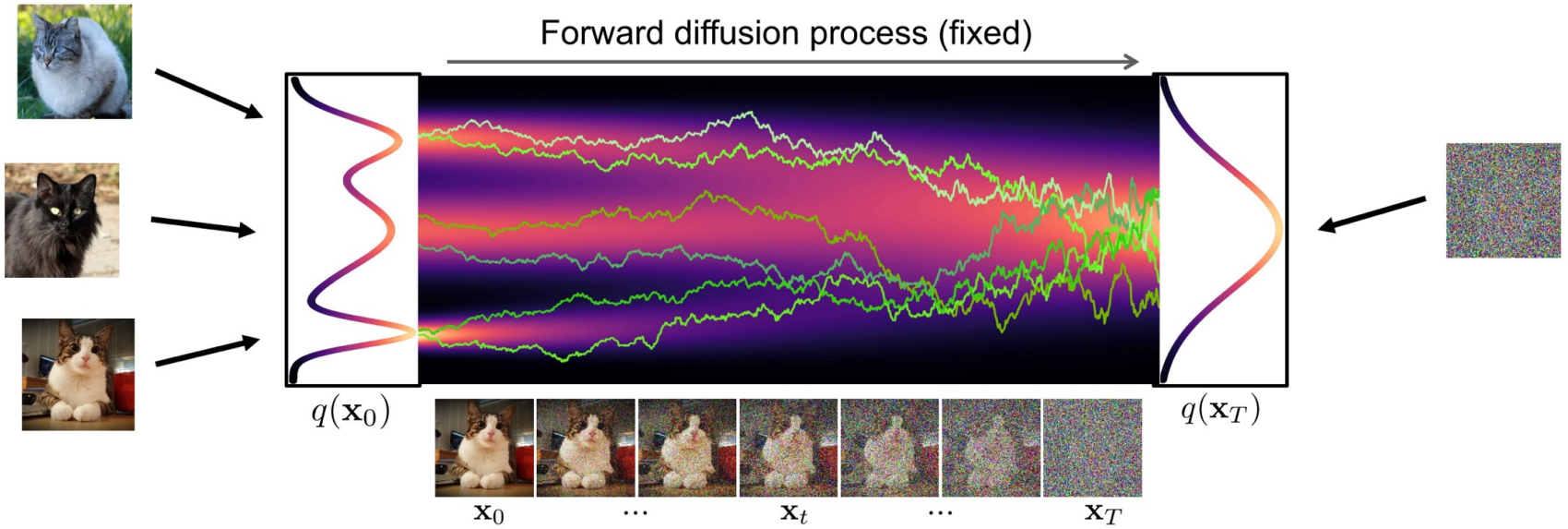
$$\mathbf{x}_t \approx \mathbf{x}_{t-1} - \frac{\beta(t)\Delta t}{2}\mathbf{x}_{t-1} + \sqrt{\beta(t)\Delta t}\,\mathcal{N}(\mathbf{0}, \mathbf{I})$$

$$\mathrm{d}\mathbf{x}_t = -\frac{1}{2}\beta(t)\mathbf{x}_t\,\mathrm{d}t + \sqrt{\beta(t)}\,\mathrm{d}\boldsymbol{\omega}_t$$

Stochastic Differential Equation (SDE)
describing the diffusion in infinitesimal limit

Song et al., "Score-Based Generative Modeling through Stochastic Differential Equations", ICLR, 2021

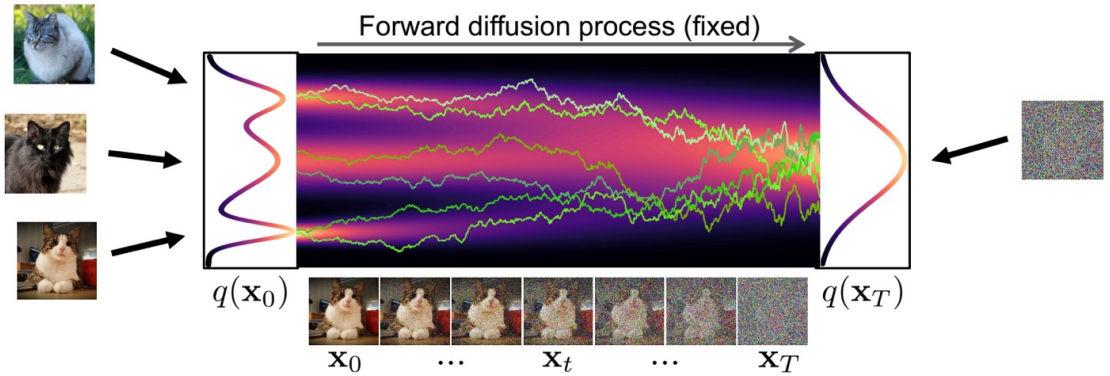
Forward Diffusion Process as Stochastic Differential Equation



Forward Diffusion SDE:

$$\mathrm{d}\mathbf{x}_t = \underbrace{-\frac{1}{2}\beta(t)\mathbf{x}_t\,\mathrm{d}t}_{\text{drift term (pulls towards mode)}} + \underbrace{\sqrt{\beta(t)}\,\mathrm{d}\boldsymbol{\omega}_t}_{\text{diffusion term (injects noise)}}$$

What about the reverse direction, necessary for generation?



Forward Diffusion SDE:

$$\mathrm{d}\mathbf{x}_t = -\frac{1}{2}\beta(t)\mathbf{x}_t\,\mathrm{d}t + \sqrt{\beta(t)}\,\mathrm{d}\boldsymbol{\omega}_t$$

Reverse Generative Diffusion SDE:

$$\mathrm{d}\mathbf{x}_t = \underbrace{\left[-\frac{1}{2}\beta(t)\mathbf{x}_t - \beta(t)\nabla_{\mathbf{x}_t}\log q_t(\mathbf{x}_t)\right]}_{\text{drift term}}\mathrm{d}t + \underbrace{\sqrt{\beta(t)}\,\mathrm{d}\bar{\boldsymbol{\omega}}_t}_{\text{diffusion term}}$$

"Score Function"

➡ **Simulate reverse diffusion process: Data generation from random noise!**

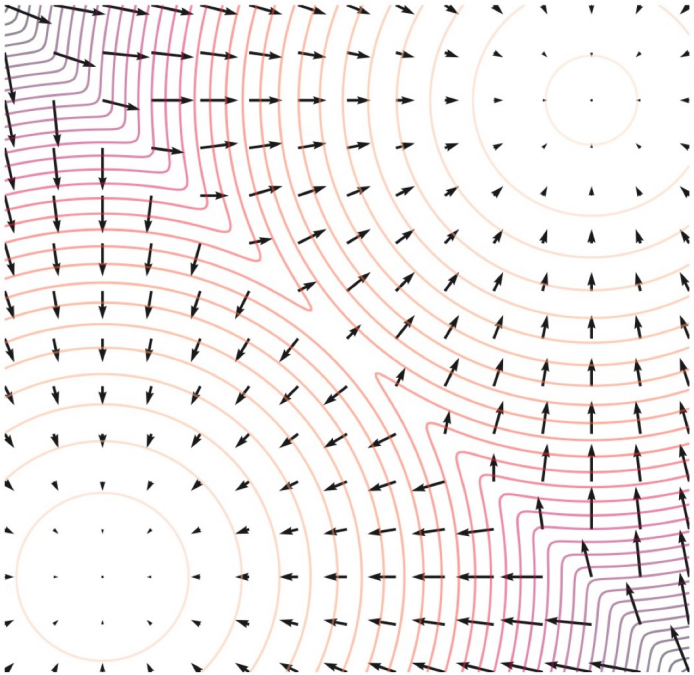
Score function

The (Stein) score function is the gradient of the log-probability of a distribution w.r.t. to the input

$$\nabla_x \log p(x)$$

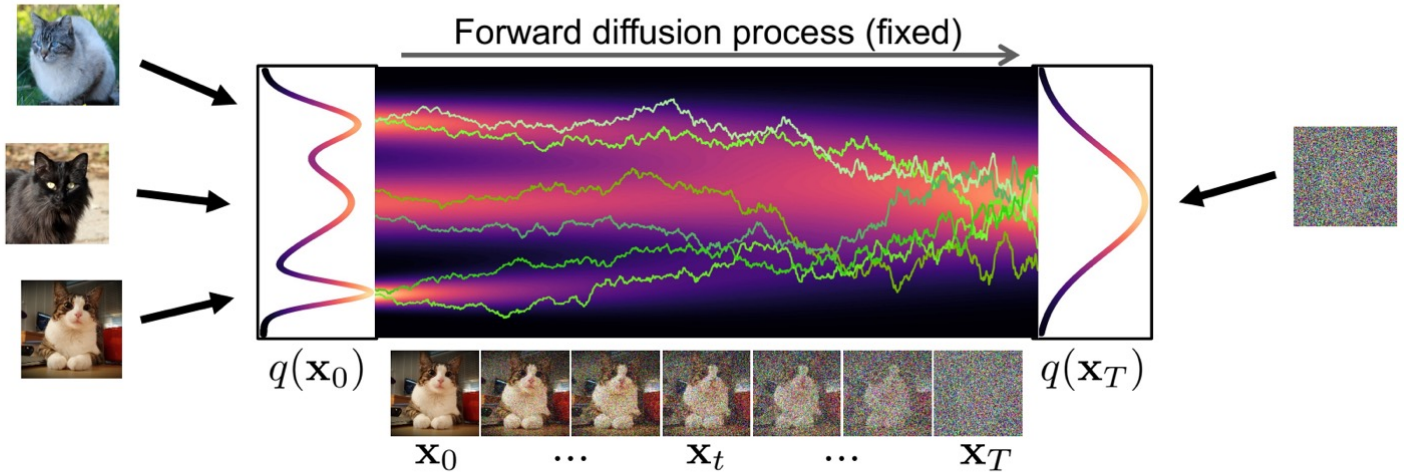
A model $s_\theta(x)$, which models the score function explicitly, is a score-based model

$$s_\theta(x) \approx \nabla_x \log p(x)$$



The score function of a mixture of two Gaussians

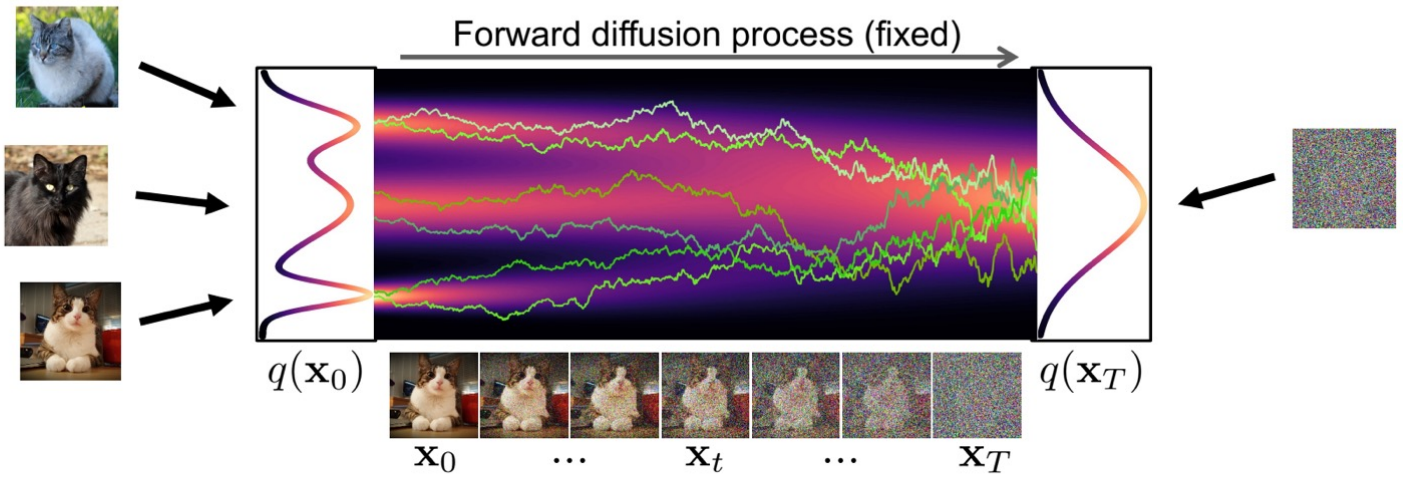
Score Matching



$$\min_{\boldsymbol{\theta}} \underbrace{\mathbb{E}_{t\sim\mathcal{U}(0,T)}}_{\text{diffusion time } t} \underbrace{\mathbb{E}_{\mathbf{x}_t\sim q_t(\mathbf{x}_t)}}_{\text{diffused data } \mathbf{x}_t} \Big|\Big| \underbrace{\mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)}_{\text{neural network}} - \underbrace{\nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t)}_{\text{score of diffused data (marginal)}} \Big|\Big|_2^2$$

But $\nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t)$ (score of the *marginal diffused density* $q_t(\mathbf{x}_t)$) is not tractable!

Denoising Score Matching



diffuse individual data points $\mathbf{x}_0$. Diffused $q_t(\mathbf{x}_t|\mathbf{x}_0)$ ***is*** tractable!

$$\min_{\boldsymbol{\theta}} \mathbb{E}_{t\sim\mathcal{U}(0,T)} \mathbb{E}_{\mathbf{x}_0\sim q_0(\mathbf{x}_0)} \mathbb{E}_{\mathbf{x}_t\sim q_t(\mathbf{x}_t|\mathbf{x}_0)} ||\mathbf{s}_{\boldsymbol{\theta}}(\mathbf{x}_t, t) - \nabla_{\mathbf{x}_t} \log q_t(\mathbf{x}_t|\mathbf{x}_0)||_2^2$$

diffusion time t	data sample $\mathbf{x}_0$	diffused data sample $\mathbf{x}_t$	neural network	score of diffused data sample

"Variance Preserving" SDE:

$$\mathrm{d}\mathbf{x}_t = -\frac{1}{2}\beta(t)\mathbf{x}_t\,\mathrm{d}t + \sqrt{\beta(t)}\,\mathrm{d}\boldsymbol{\omega}_t$$

$$q_t(\mathbf{x}_t|\mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \gamma_t\mathbf{x}_0, \sigma_t^2\mathbf{I})$$

$$\gamma_t = e^{-\frac{1}{2}\int_0^t \beta(s)ds}$$

$$\sigma_t^2 = 1 - e^{-\int_0^t \beta(s)ds}$$

Applications

Jason Allen’s AI-generated work, “Théâtre D’opéra Spatial,” won first place in the digital category at the Colorado State Fair.

Text-to-Image Generation

DALL·E 2

"a teddy bear on a skateboard in times square"

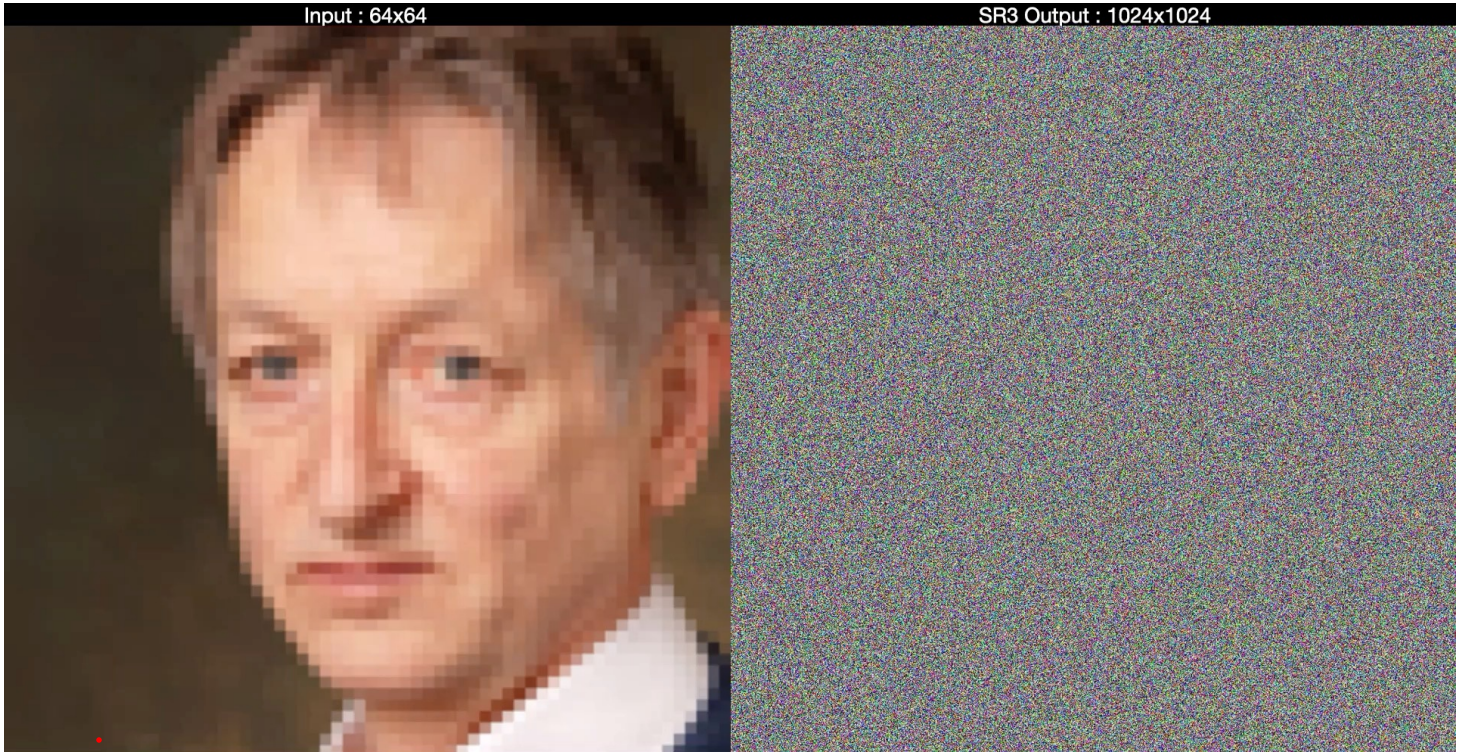
"Hierarchical Text-Conditional Image Generation with CLIP Latents" Ramesh et al., 2022

Imagen

A group of teddy bears in suit in a corporate office celebrating the birthday of their friend. There is a pizza cake on the desk.

"Photorealistic Text-to-Image Diffusion Models with Deep Language Understanding", Saharia et al., 2022

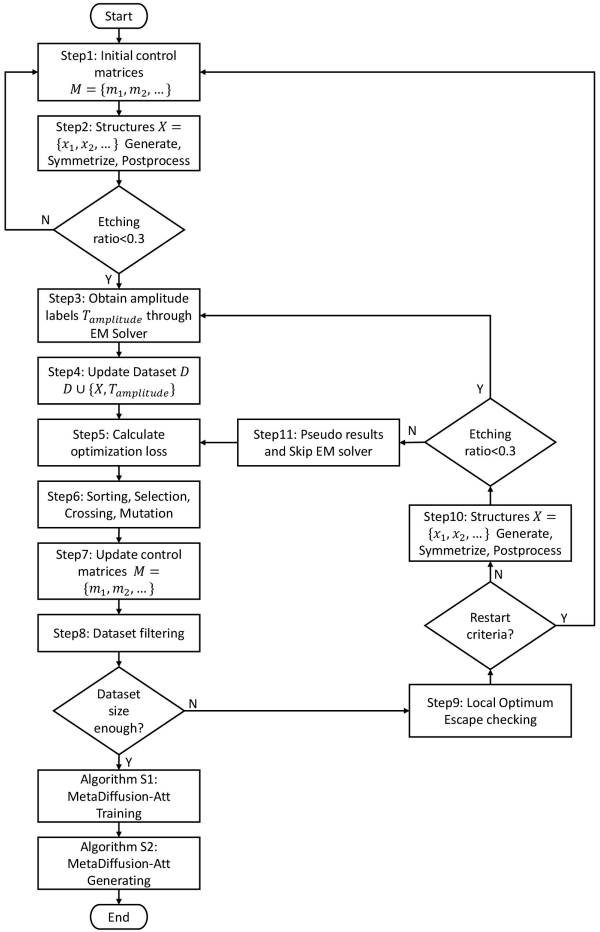
Image Super-resolution

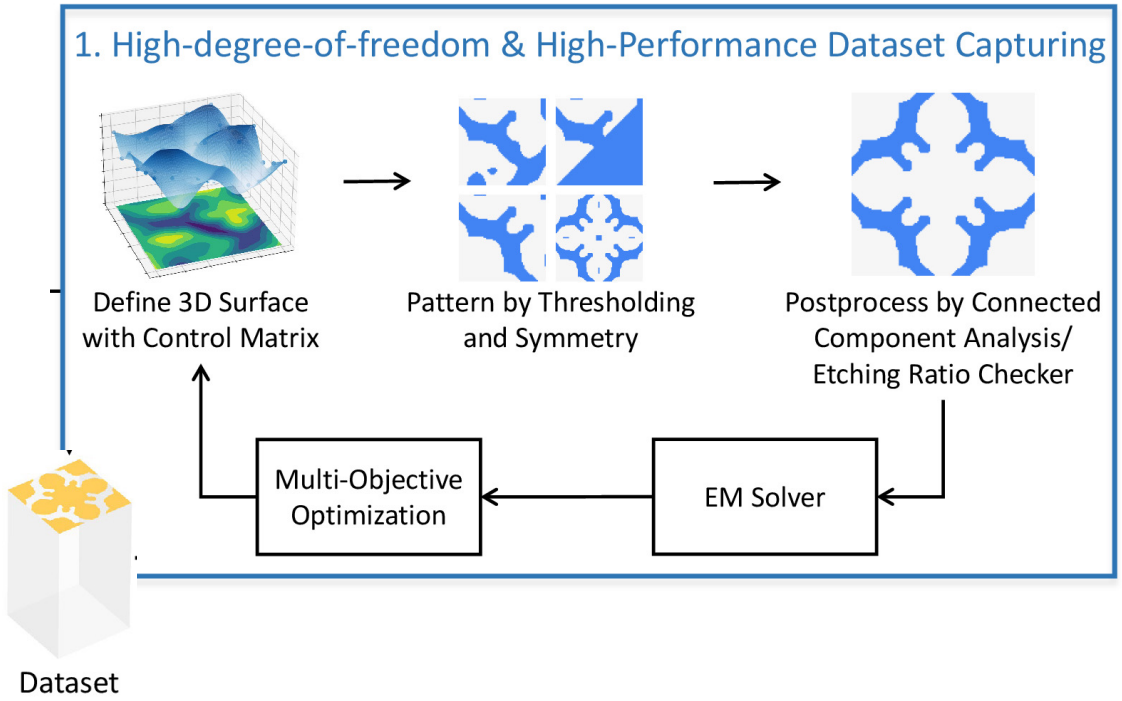


https://iterative-refinement.github.io

metasurface inverse design

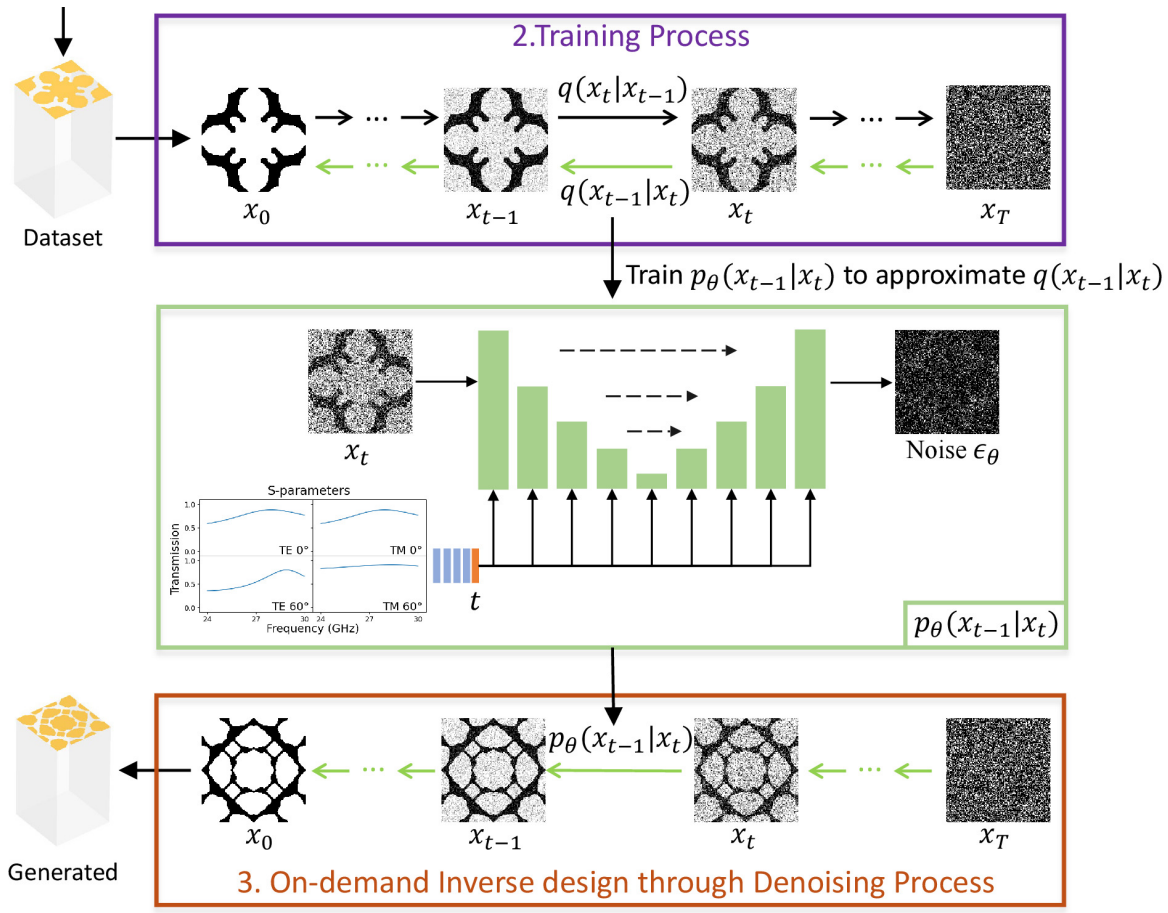
https://doi.org/10.1364/OE.537389

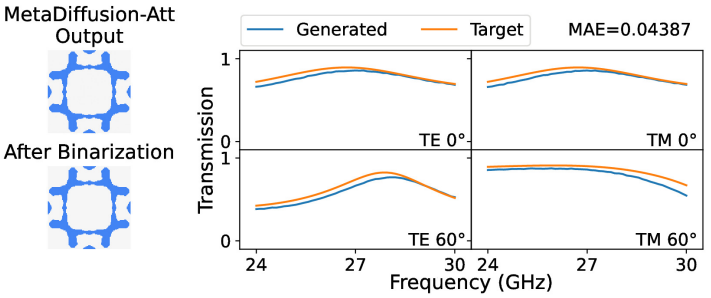


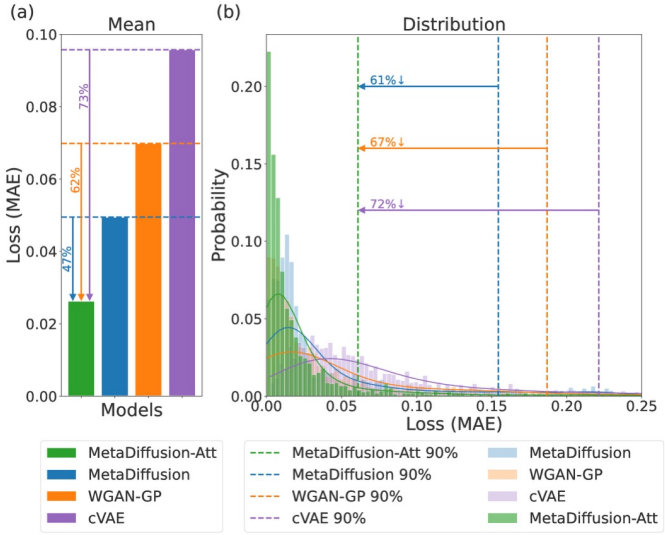


metasurface inverse design

https://doi.org/10.1364/OE.537389

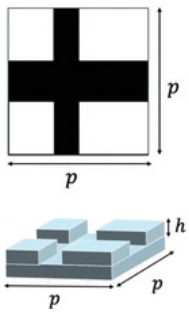


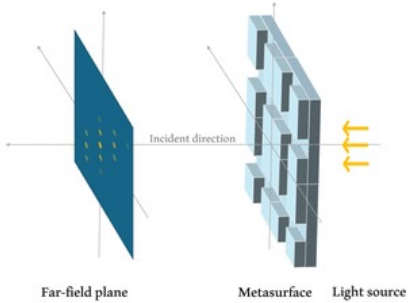


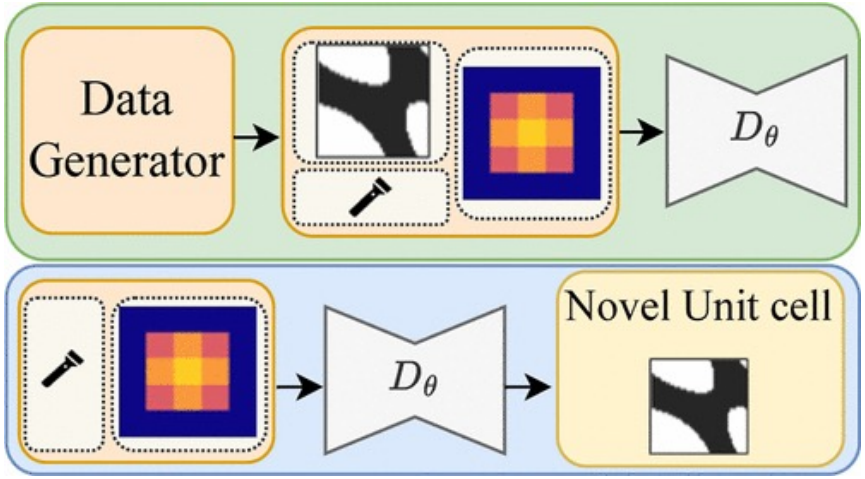


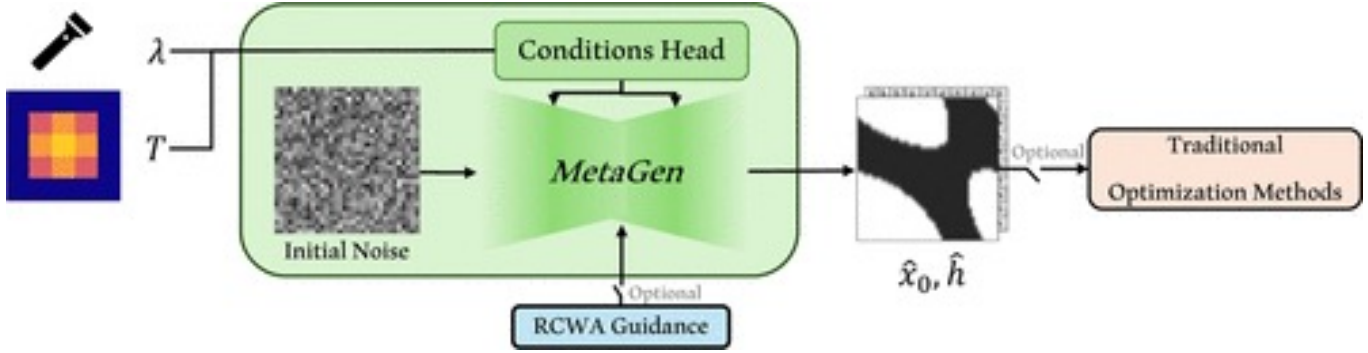
metasurface inverse design

https://doi.org/10.1021/acsphotonics.5c01384









Quantum circuit synthesis with diffusion models

https://doi.org/10.1038/s42256-024-00831-9

I want a circuit that ...

... compiles this unitary

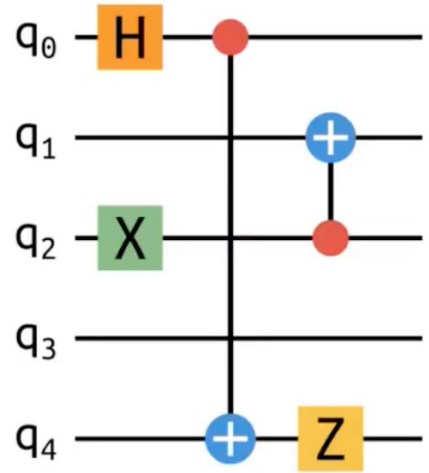
... prepares this entangled state

$$U = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

ML model

generate SRV: [1, 2, 2]

compile using: [h, cx, x]



Quantum circuit synthesis with diffusion models

Cuda-Q tutorial NVIDIA

genQC diffusion model based quantum circuit synthesis

1. Robust

strong performance

avoids quantum complexity in training

2. Scalable

same trained model, any size or length

train on easy examples, finetune to complex cases

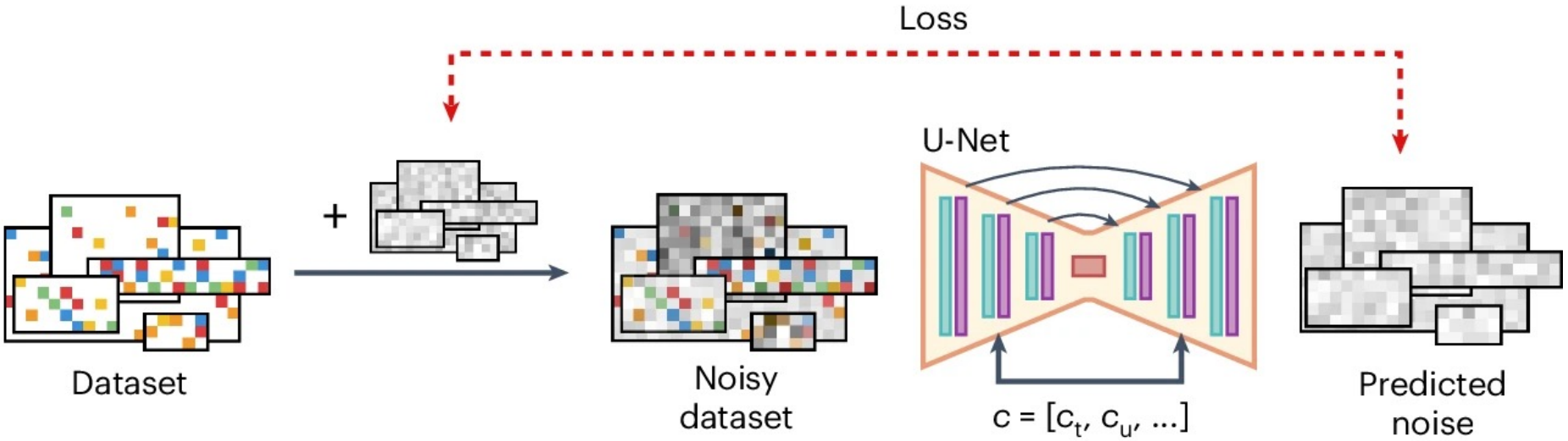
3. Flexible

text prompting sets the goal

inputs can be tweaked to mimic physical constraints

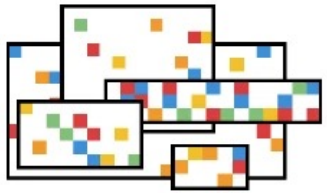
Quantum circuit synthesis with diffusion models

Training



Quantum circuit synthesis with diffusion models

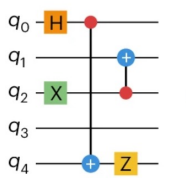
tensor representation of some circuits

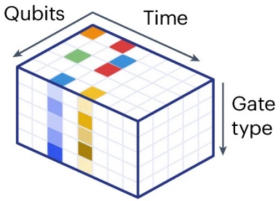


Dataset

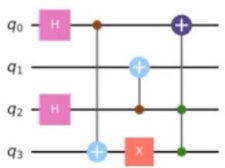
each circuit has some characteristic C_i (text prompt)

Circuit encoding





Vocabulary: empty ⇒ 0, h ⇒ 1, cx ⇒ 2, x ⇒ 3, ccx ⇒ 4, padding ⇒ 5

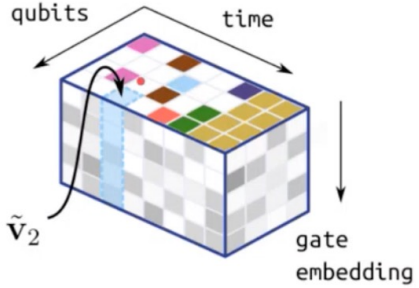


tokenize

tokenized circuit:

$$\begin{bmatrix} 1 & 0 & -2 & 0 & 0 & 4 & 5 & 5 \\ 0 & 0 & 0 & 2 & 0 & 0 & 5 & 5 \\ 0 & 1 & 0 & -2 & 0 & -4 & 5 & 5 \\ 0 & 0 & 2 & 0 & 3 & -4 & 5 & 5 \end{bmatrix}$$

embedding



cosine similarity

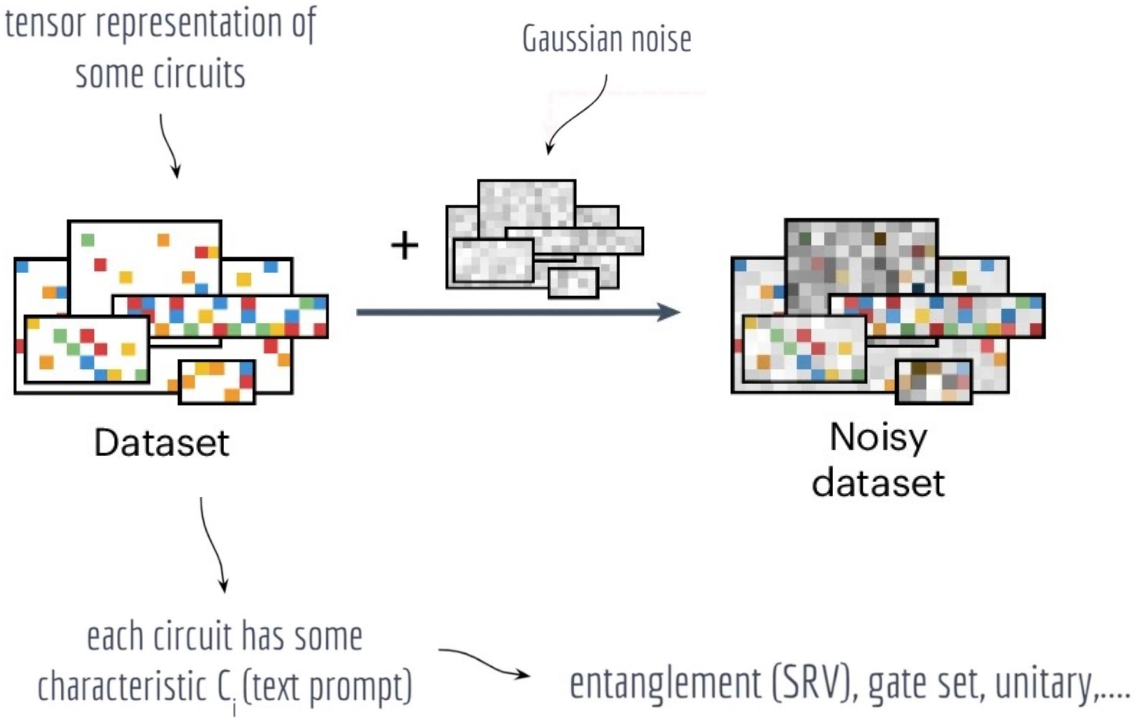
$S_C(\mathbf{v}_1, \tilde{\mathbf{v}}_2)$
$S_C(\mathbf{v}_2, \tilde{\mathbf{v}}_2)$
⋮

tokenized circuit:

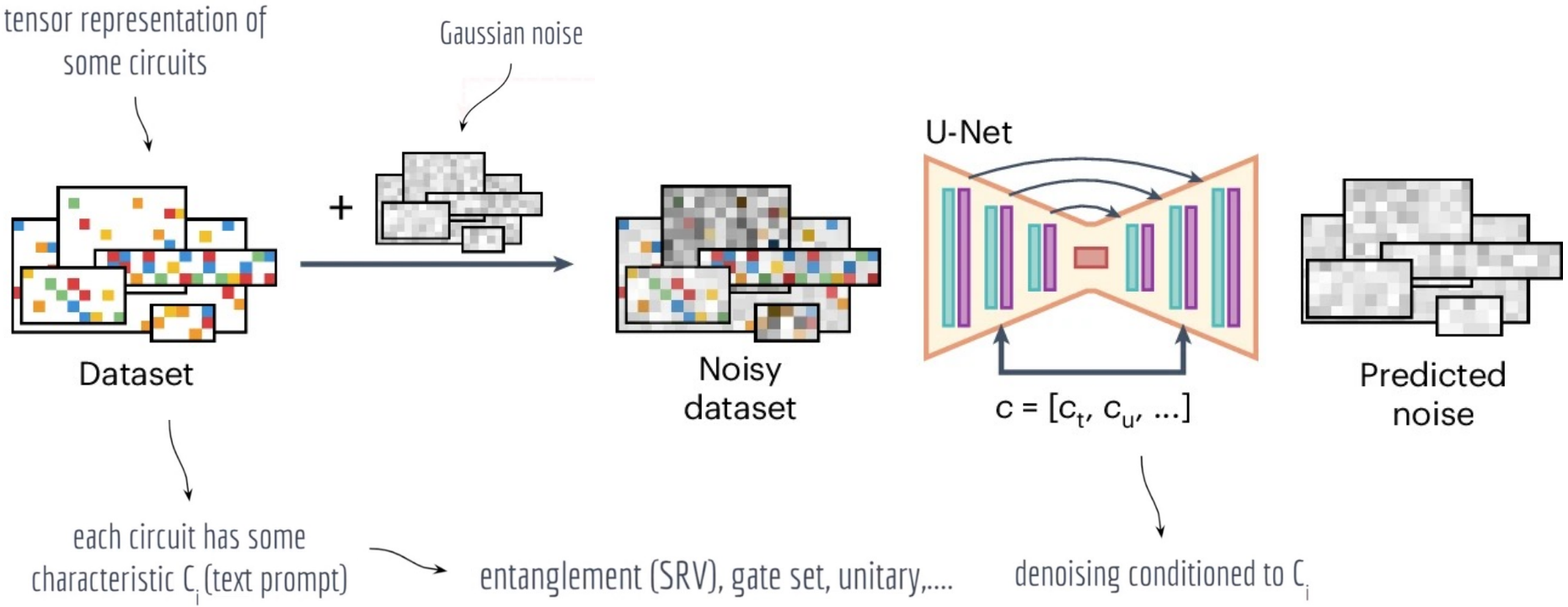
$$\begin{bmatrix} \cdots\cdots\cdots \\ \cdots\cdots\cdots \\ \cdots\cdots\cdots \end{bmatrix}$$

circuit generator

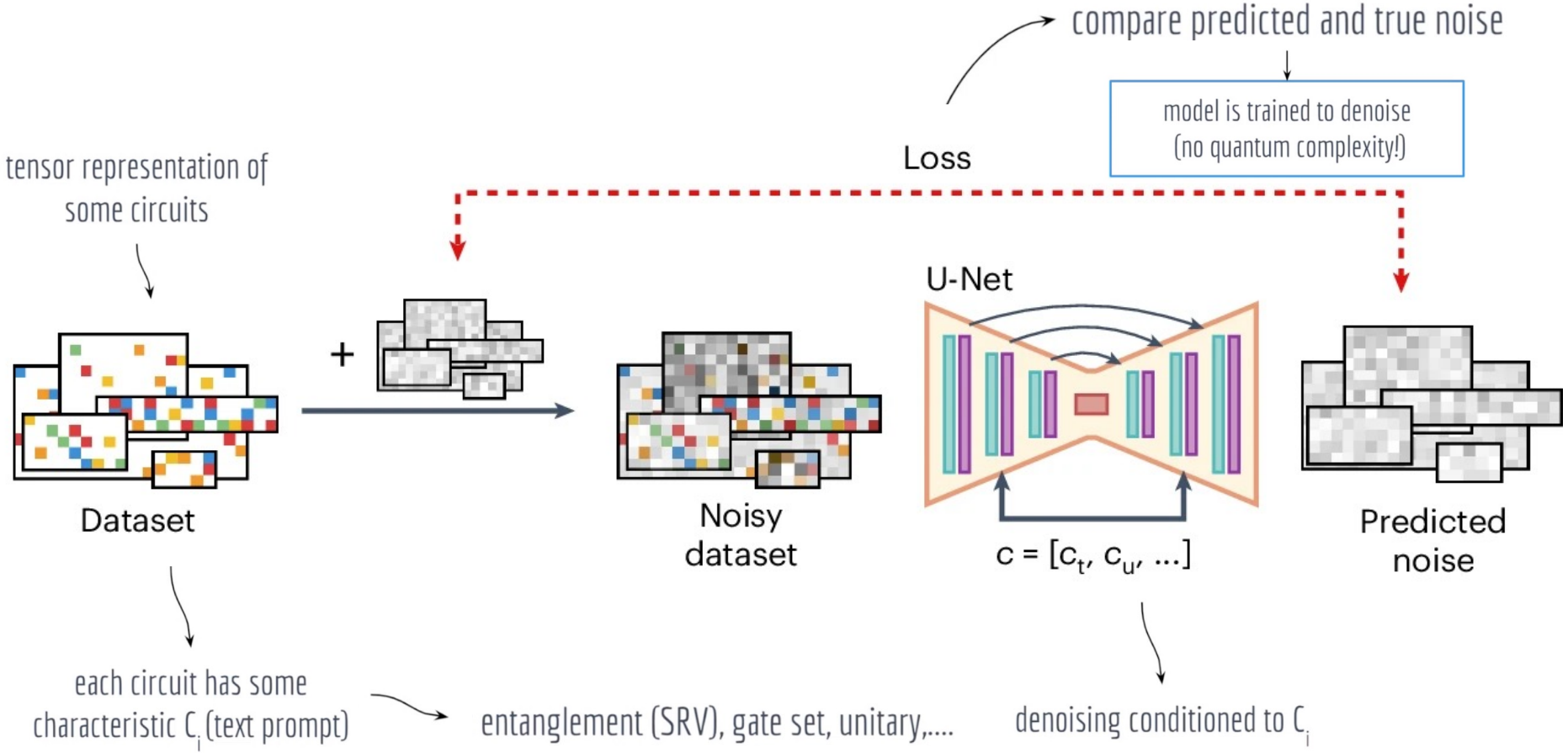
Quantum circuit synthesis with diffusion models



Quantum circuit synthesis with diffusion models

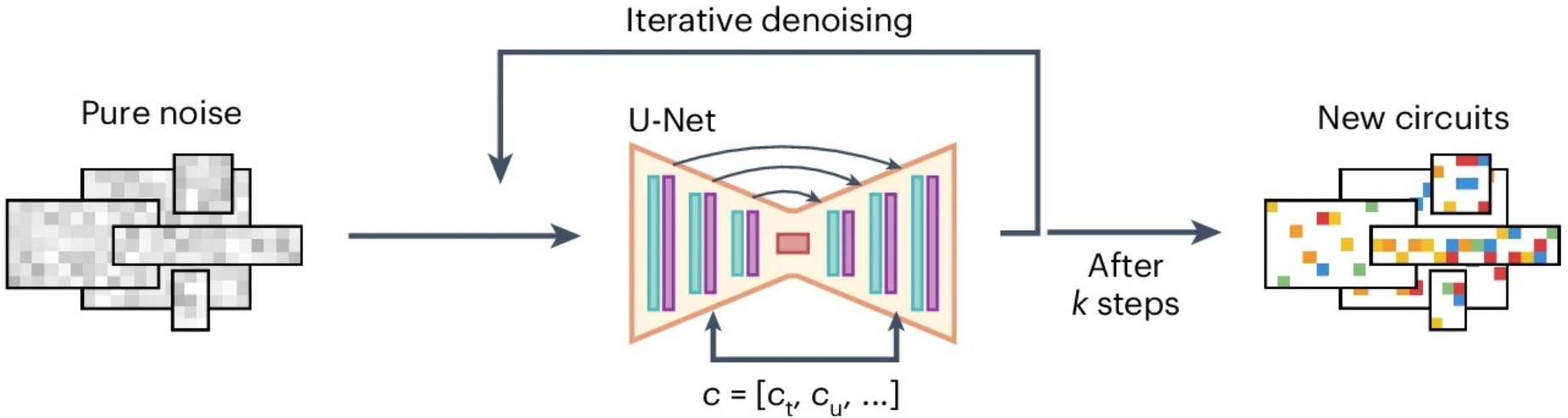


Quantum circuit synthesis with diffusion models

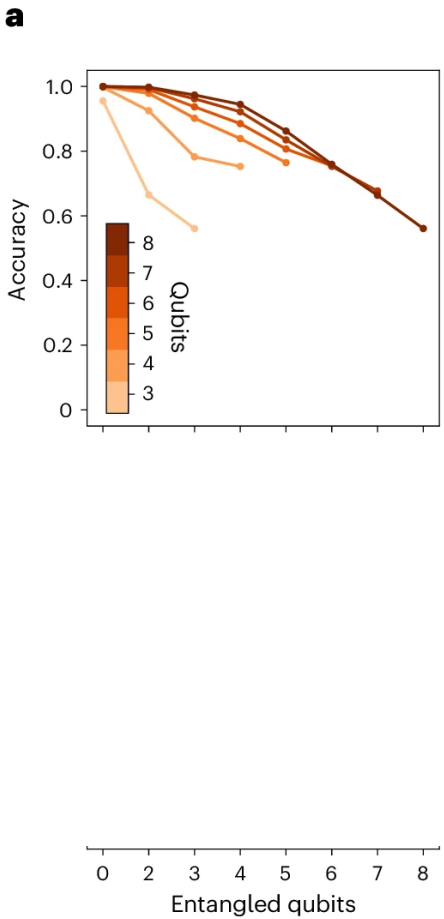


Quantum circuit synthesis with diffusion models

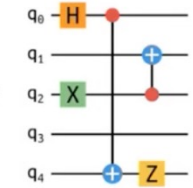
Inference ⟶ **generating new circuits**



Quantum circuit synthesis with diffusion models



generate SRV: [1, 2, 2] ⇨ genQC ⇨



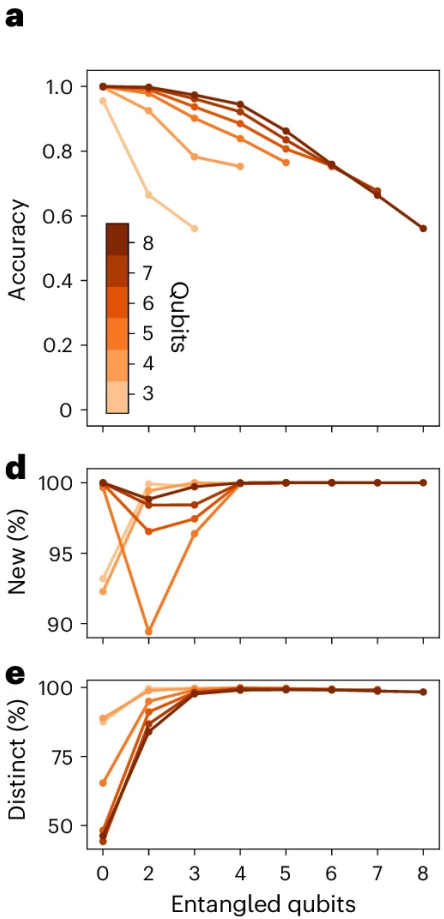
model trained on 3 to 8 qubits

Generally accurate across all qubits sizes / entanglements.

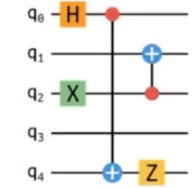
Accuracy related to: complexity and training dataset balance.

When failing, we can rely on more sampling (cheap)!

Quantum circuit synthesis with diffusion models



generate SRV: [1, 2, 2] ⇨ genQC ⇨

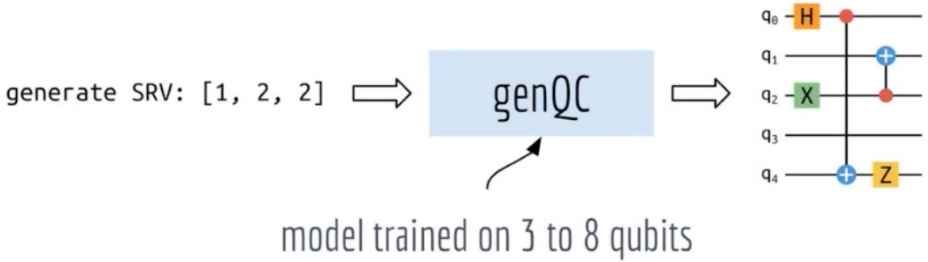


model trained on 3 to 8 qubits

The machine generalizes to unseen circuits (discovery)

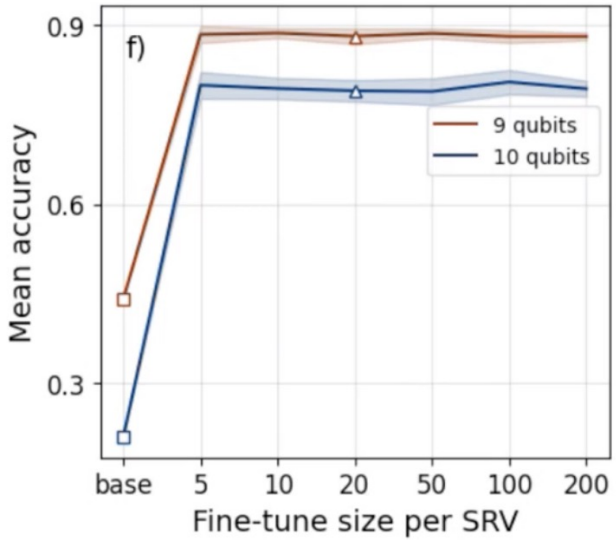
There is variety in the generated data!

Quantum circuit synthesis with diffusion models



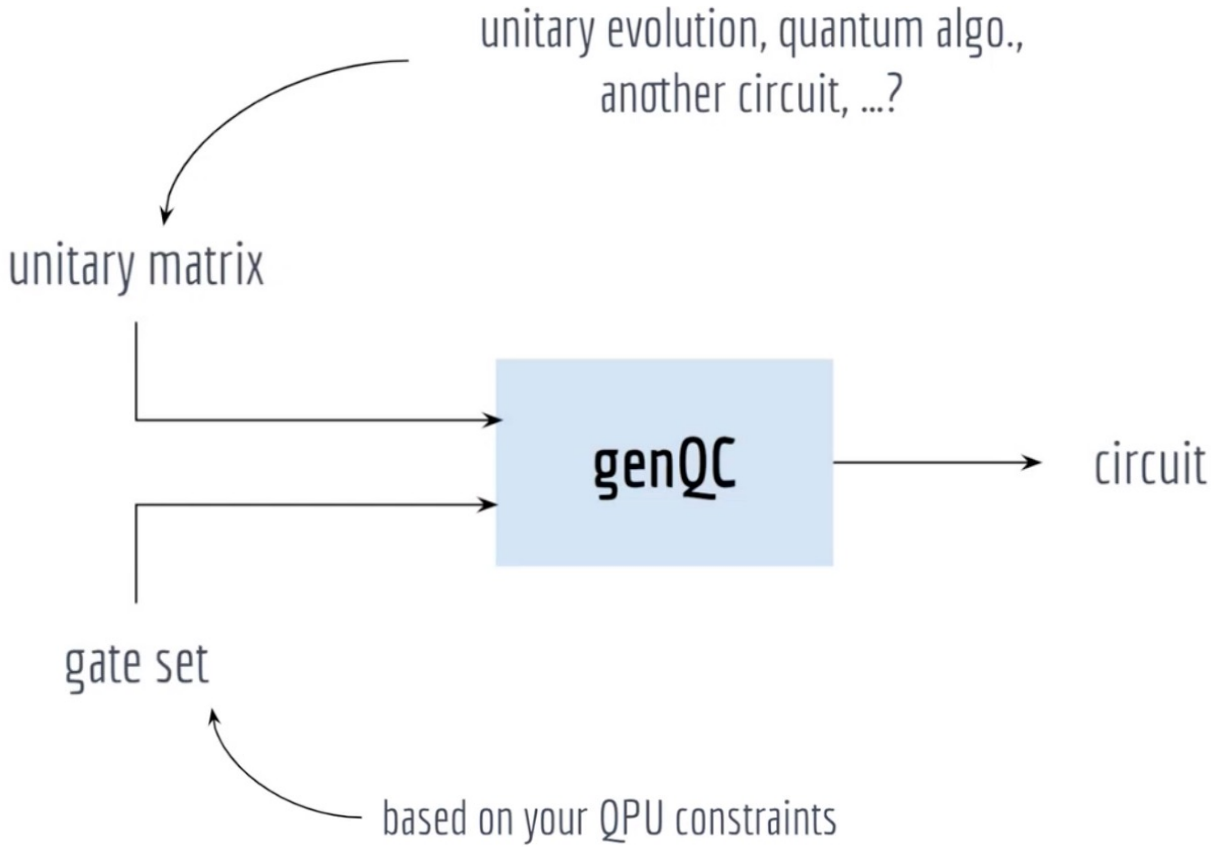
Scaling: what if the data is expensive to generate (e.g., many qubits)?

fine tuning pre-trained models ⇨

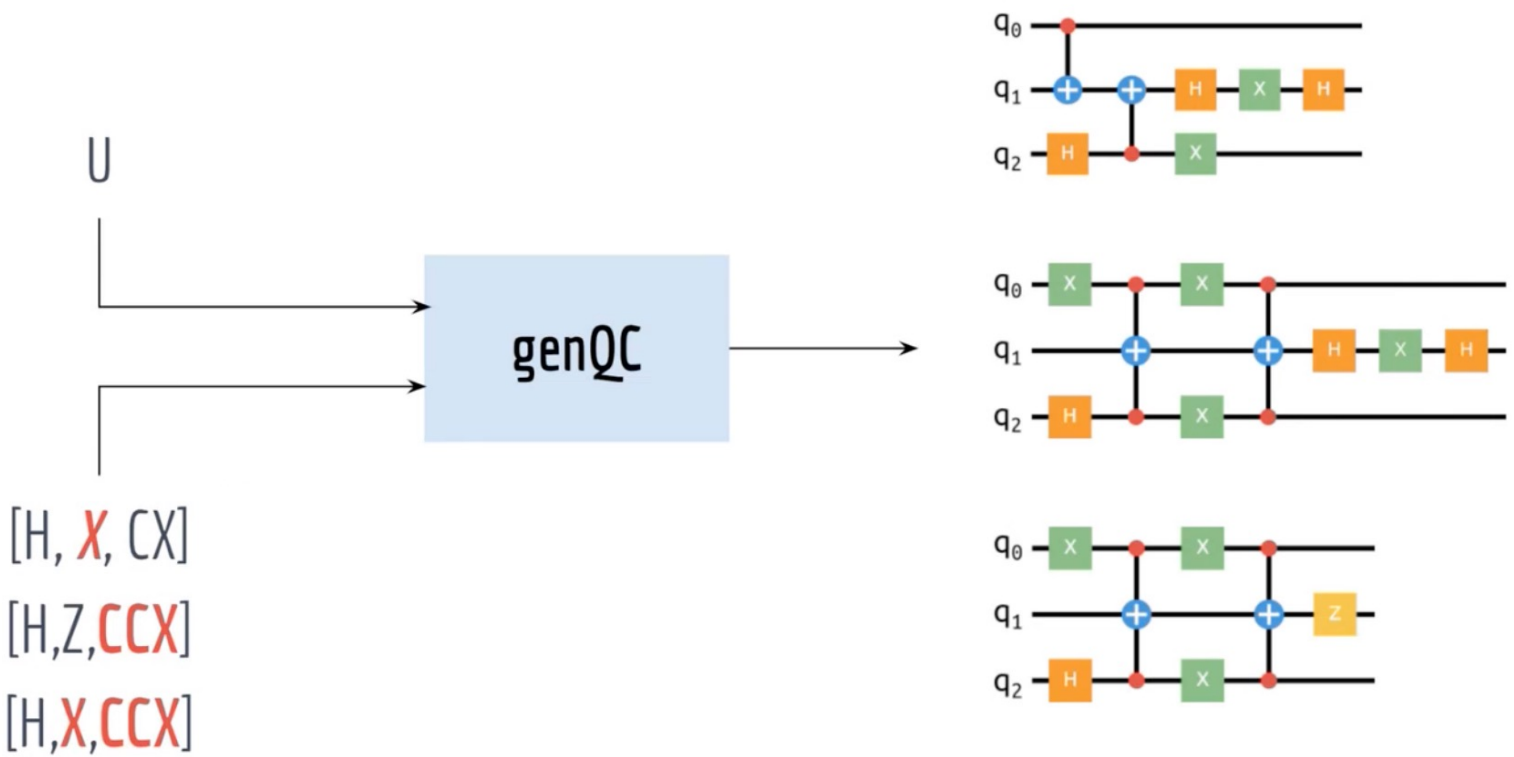


compare to the ›10k used for training!

Quantum circuit synthesis with diffusion models

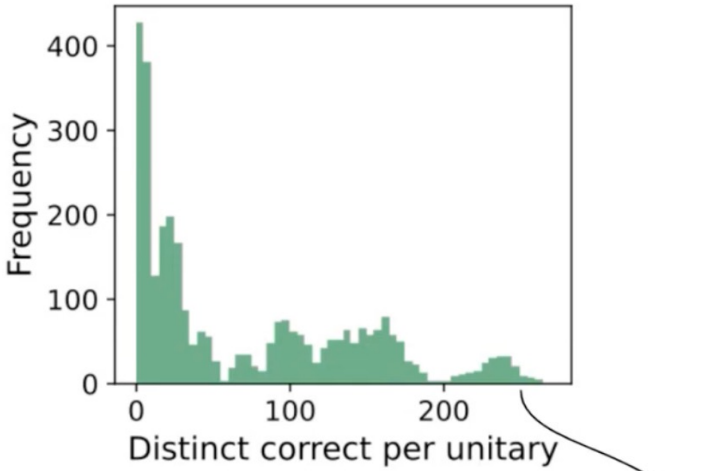


Quantum circuit synthesis with diffusion models

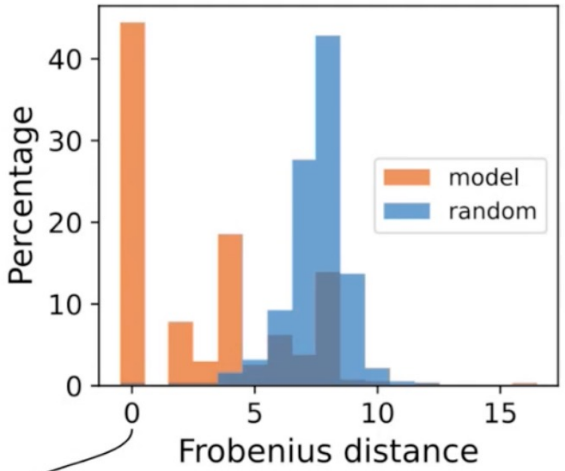


Quantum circuit synthesis with diffusion models

Exact unitary compilation (3qbits): test on 3100 random unitaries (sample 1000 circuits each)



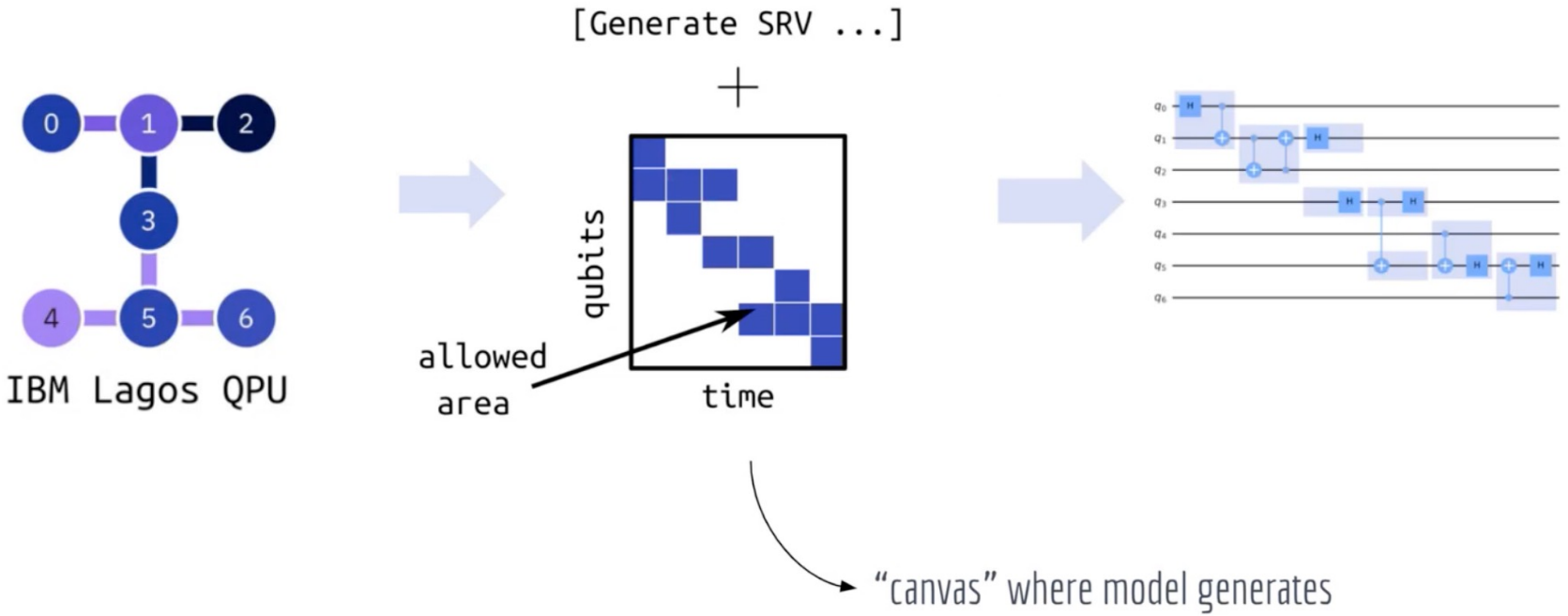
some U more than
200 valid circuits!



92.6% found the exact compilation!

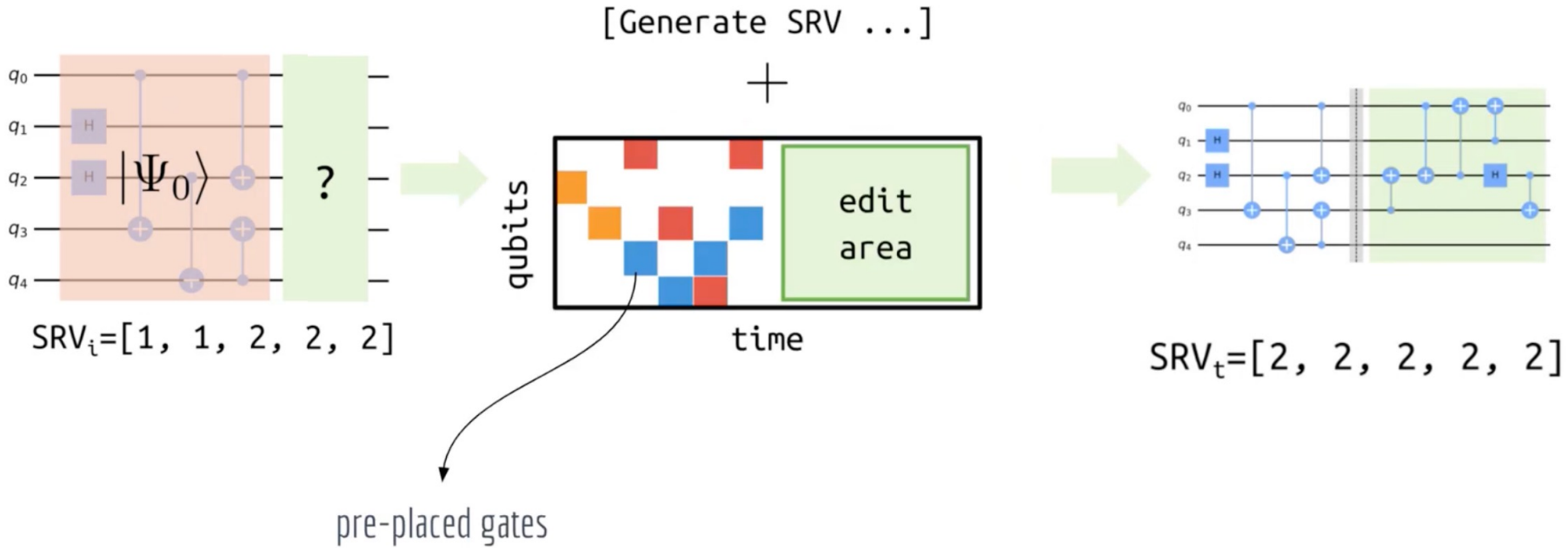
Quantum circuit synthesis with diffusion models

Masking: accounting for qubit connectivity

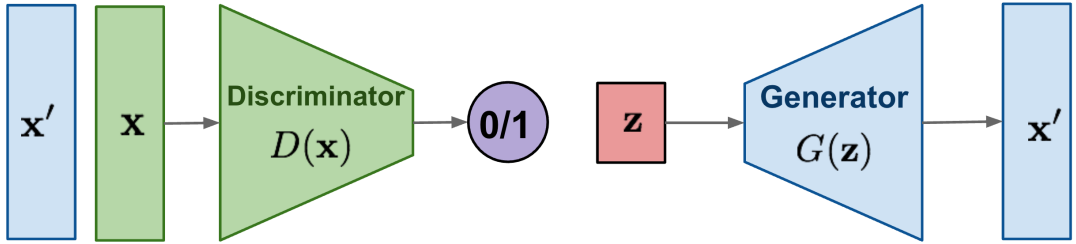


Quantum circuit synthesis with diffusion models

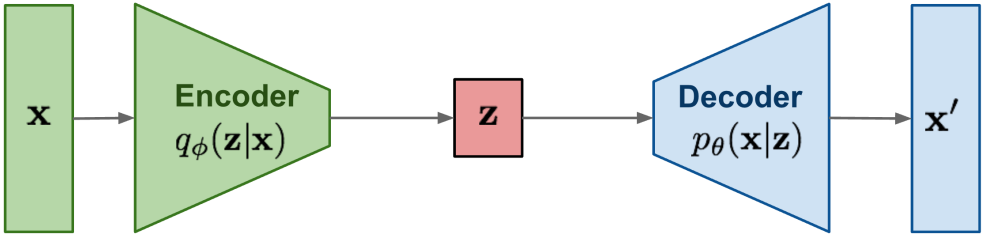
Editing circuits: given input state, generate target state



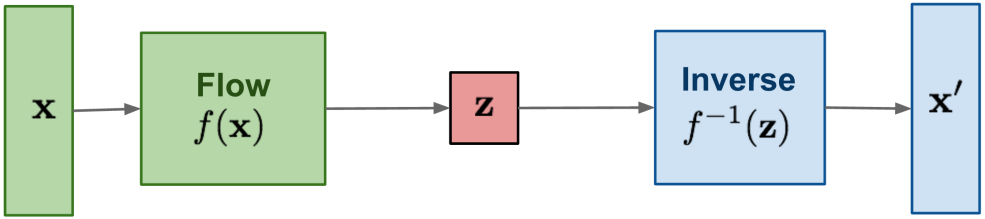
GAN: Adversarial training



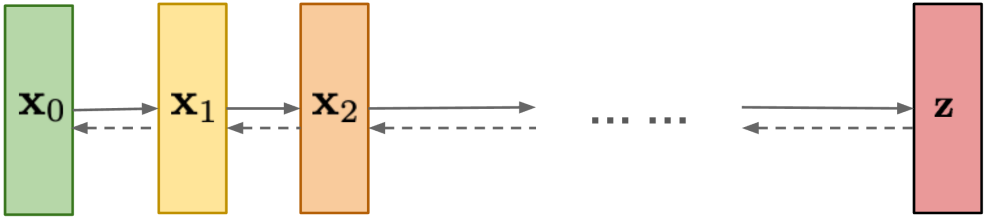
VAE: maximize variational lower bound



Flow-based models: Invertible transform of distributions

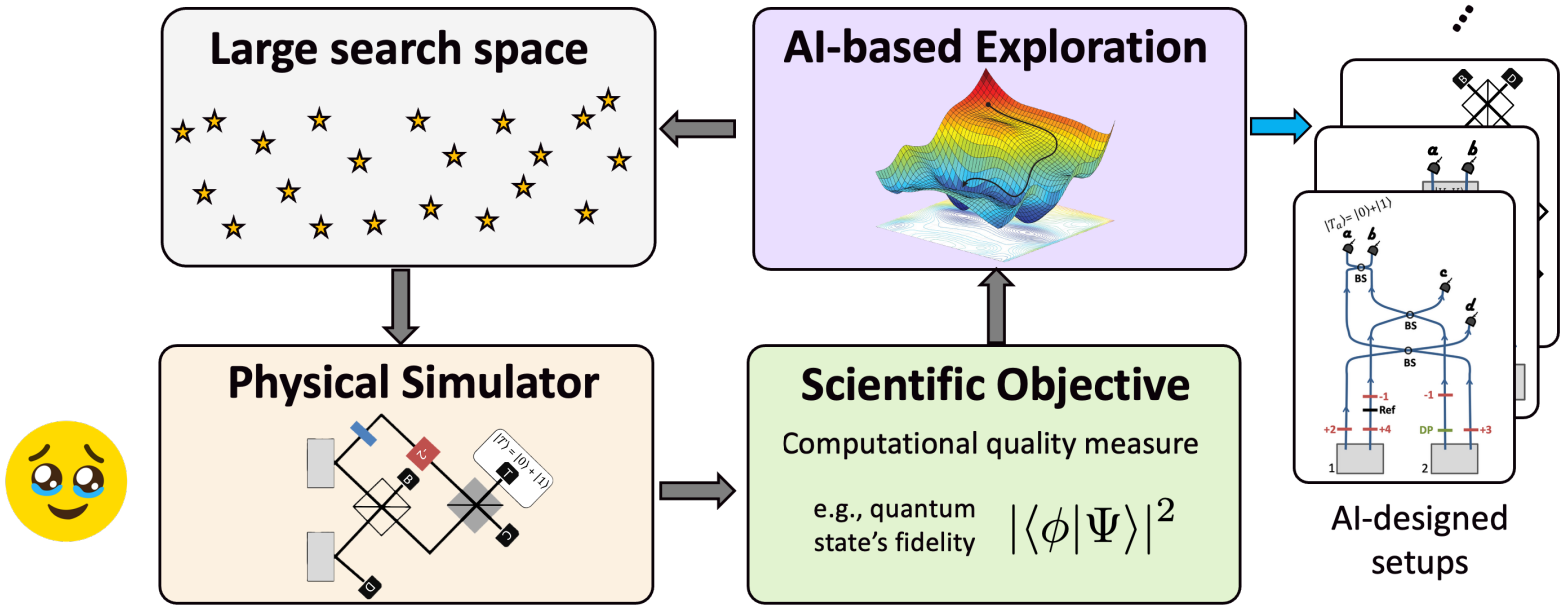


Diffusion models: Gradually add Gaussian noise and then reverse



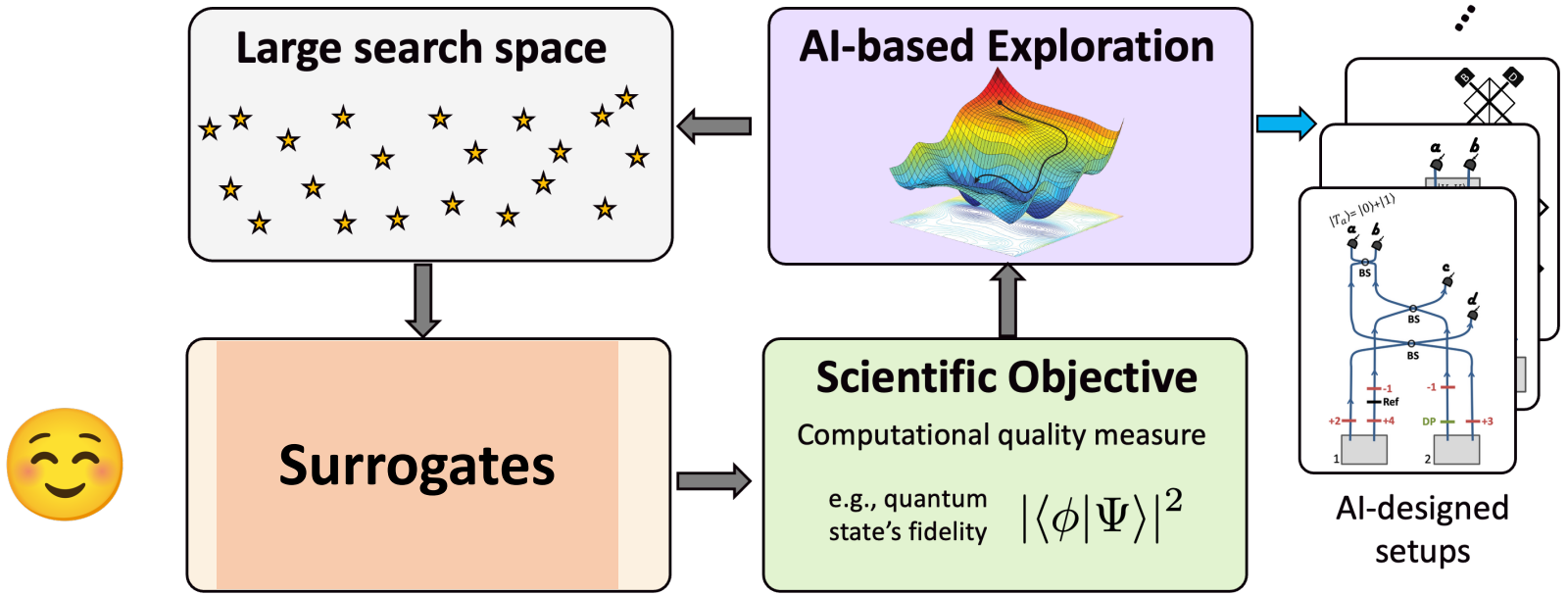
Surrogate model

a fast, simplified mathematical approximation of a complex, computationally expensive system



Surrogate model

a fast, simplified mathematical approximation of a complex, computationally expensive system

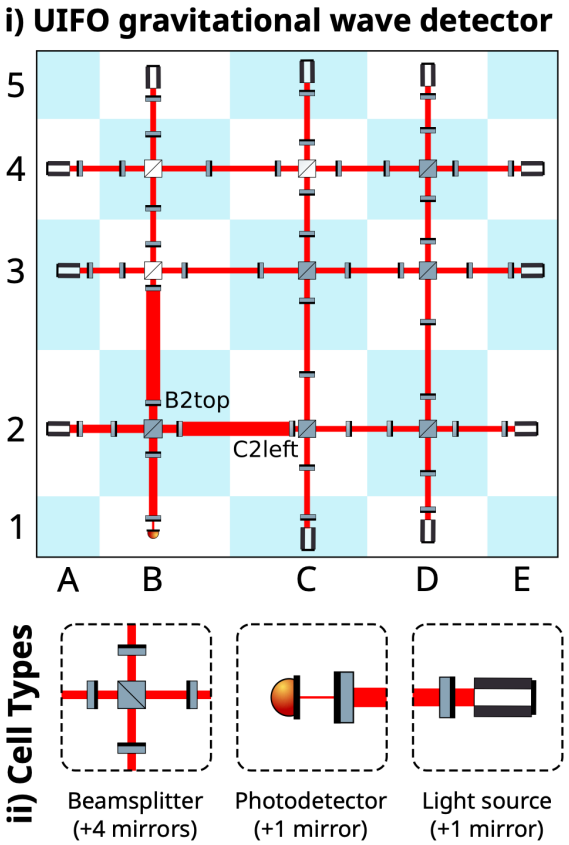


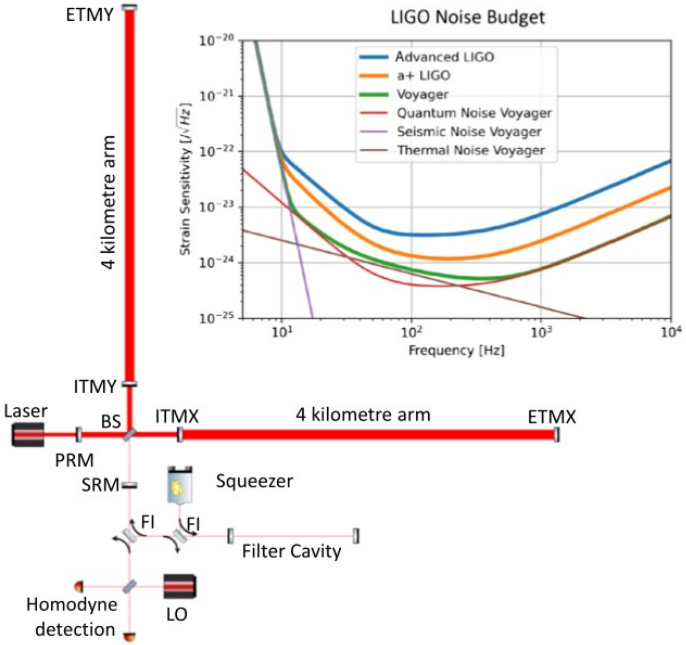
Neural surrogates for designing gravitational wave detectors

https://arxiv.org/abs/2511.19364

Finesse 3

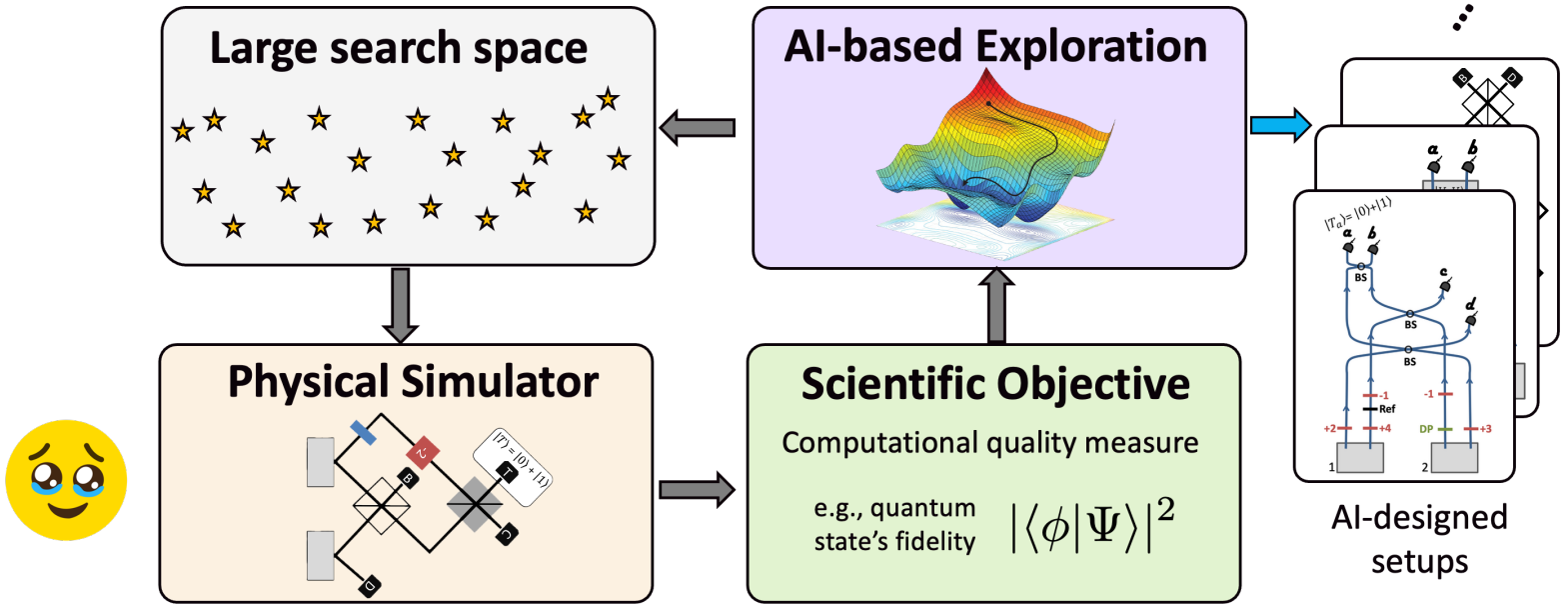
Finesse, a computationally expensive simulator





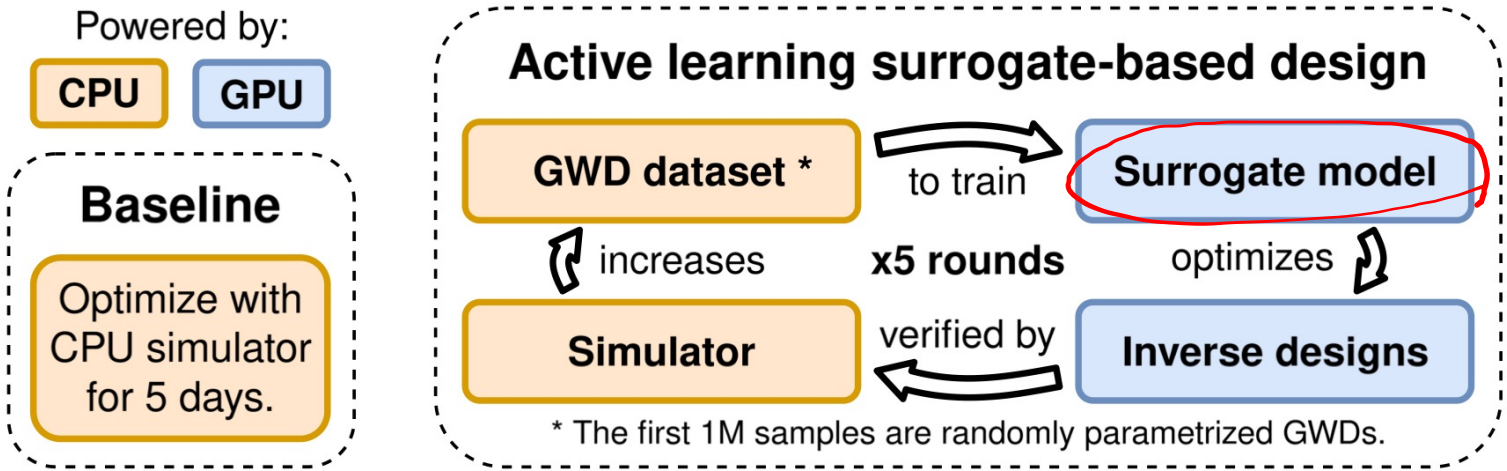
Neural surrogates for designing gravitational wave detectors

https://arxiv.org/abs/2511.19364



Neural surrogates for designing gravitational wave detectors

https://arxiv.org/abs/2511.19364

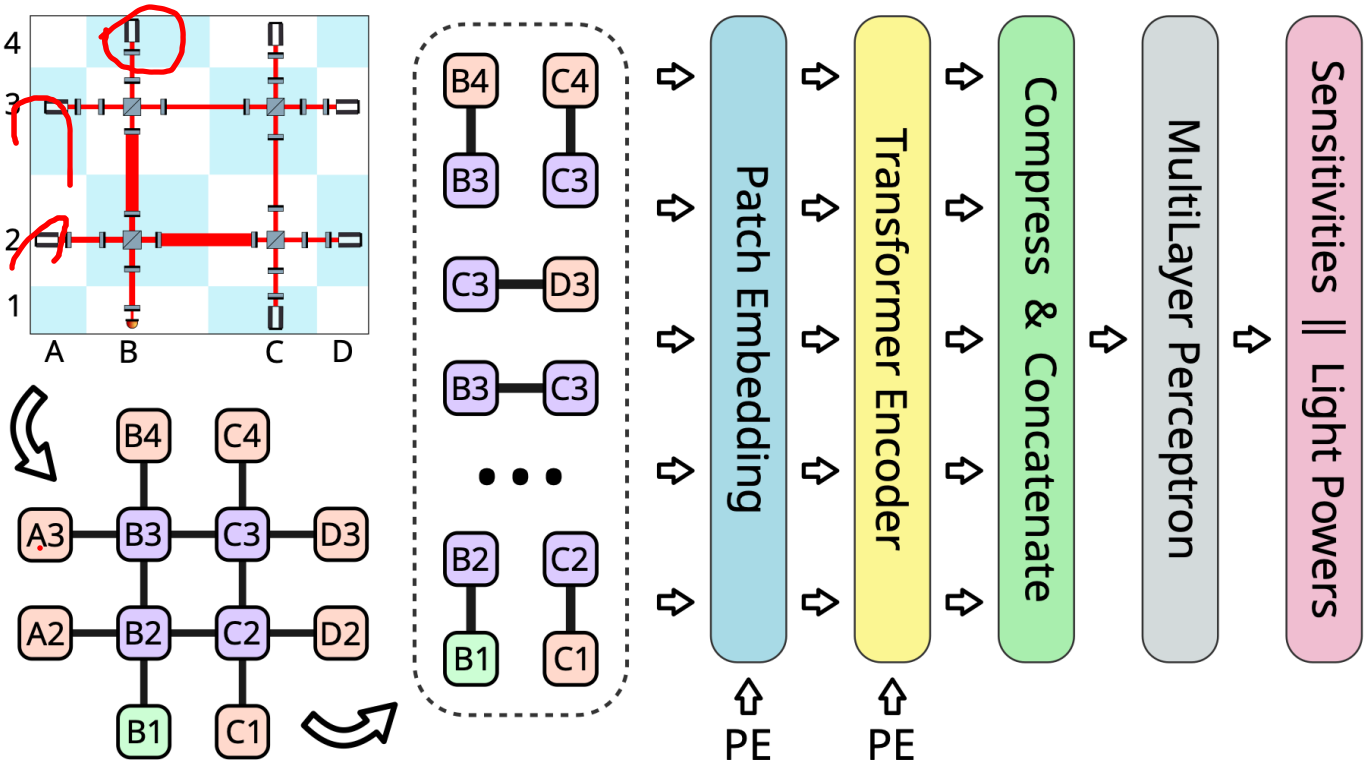


For the same quality, the surrogate-based design (+ random sampling) takes much less time than only using the simulator

Experiment size	1x1 (29 parameters)		2x2 (85 parameters)		3x3 (169 parameters)	
Design strategy	Simulator	Surrogate	Simulator	Surrogate	Simulator	Surrogate
Processor unit	CPU only	CPU+GPU	CPU only	CPU+GPU	CPU only	CPU+GPU
Time until 1e-20	1h10'	14'+40'	6h05'	41'+1h09'	32h15'	1h23'+2h01'
Simulation calls to reach 1e-20	5.5M	2.2M	9.6M	2.2M	25.6M	2.2M

Neural surrogates for designing gravitational wave detectors

https://arxiv.org/abs/2511.19364



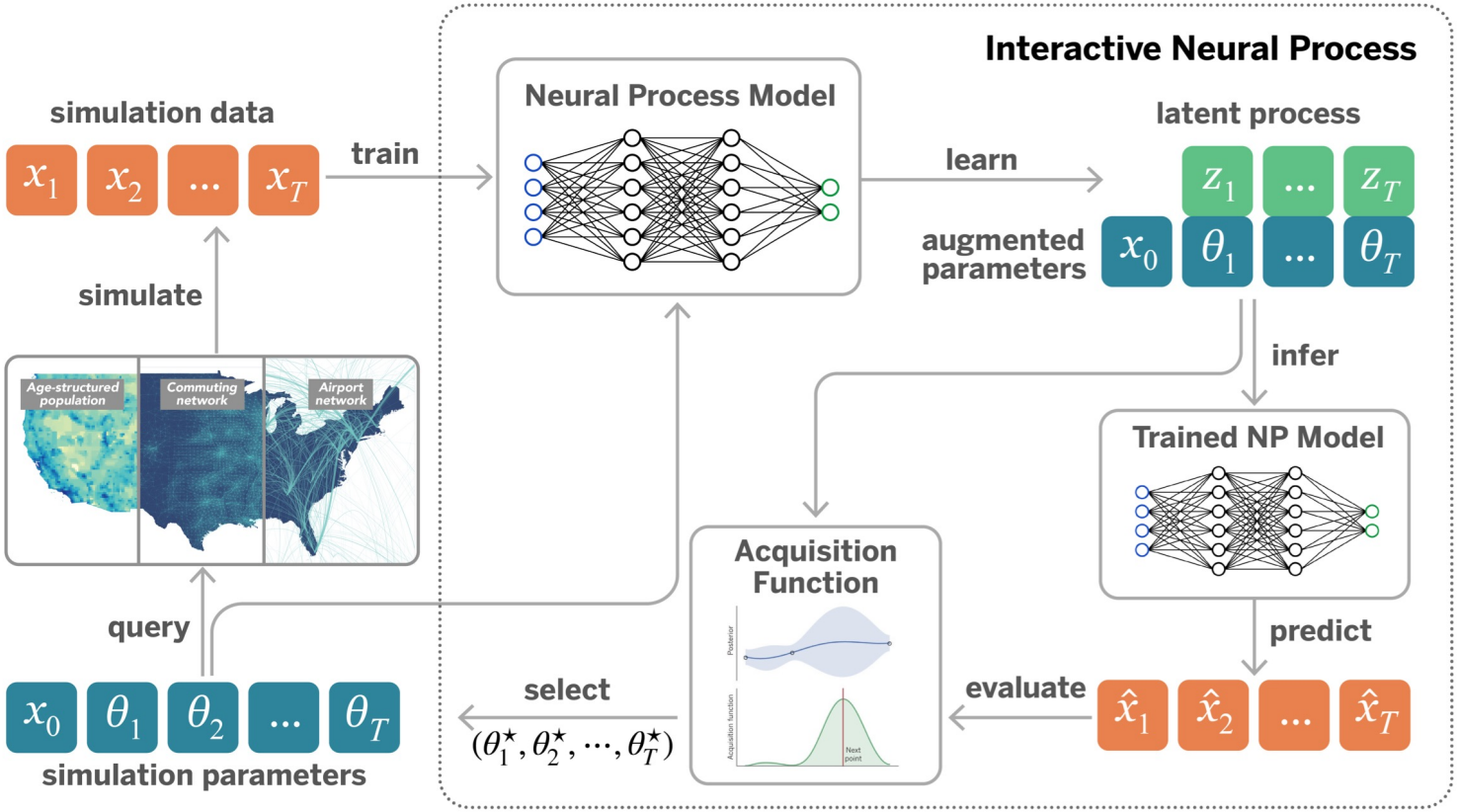
Neural surrogates for designing gravitational wave detectors

https://arxiv.org/abs/2511.19364

UIFO grid size	1x1 (29 parameters)		2x2 (85 parameters)		3x3 (169 parameters)	
Design tool	Simulator	Surrogate	Simulator	Surrogate	Simulator	Surrogate
Simulator calls	560.6M	6.6M	189.3M	6.6M	95.2M	6.6M
Final samples	256	1.1M	256	1.1M	256	1.1M
Steps / sample	73001	16384	8600	16384	2189	16384
Best sensitivity	2.25e-24	1.07e-23	2.85e-21	7.56e-24	1.68e-21	7.92e-24
Time spent	CPU	CPU+GPU	CPU	CPU+GPU	CPU	CPU+GPU
Until 1e-20	1h10'	14'+40'	6h05'	41'+1h09'	32h15'	1h23'+2h01'
Until 1e-22	5h09'	28'+1h44'	90h13'	1h23'+2h56'	Never	2h46'+4h37'
Total	5 days	1h24'+7h36'	5 days	4h10'+11h27'	5 days	8h18'+18h6'

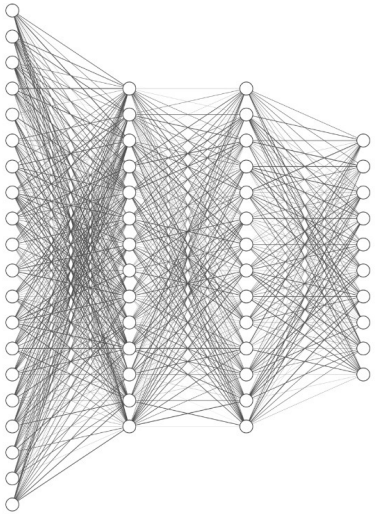
Accelerating Stochastic Simulation

https://arxiv.org/abs/2106.02770



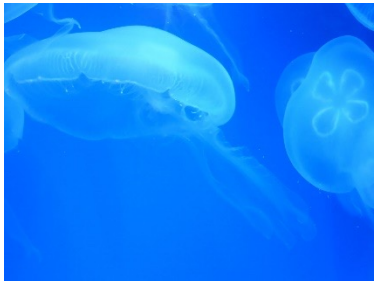
Deep Dreaming

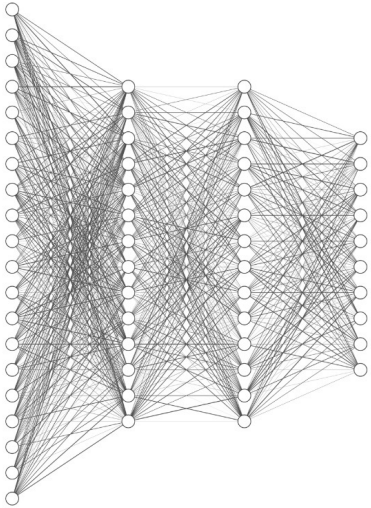
DeepDream is a computer vision program created by Google engineer Alexander Mordvintsev that uses a convolutional neural network to find and enhance patterns in images via algorithmic pareidolia, thus creating a dream-like appearance reminiscent of a psychedelic experience in the deliberately overprocessed images



0.11 black widow
0.08 cockroach
0.91 cat
0.05 golfcart
0.01 dog
0.02 mite
0.01 moon jelly
…

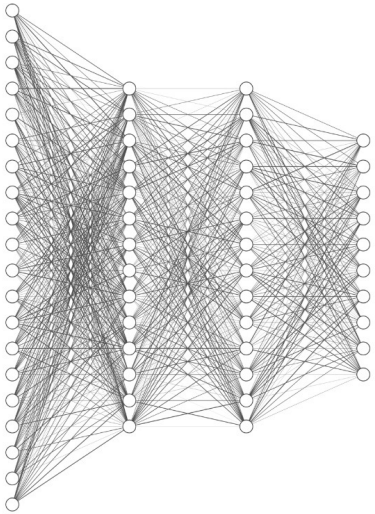
$$L = \sum_{j=0}^{neurons} \left(a_j^{(L)} - y_j \right)^2$$





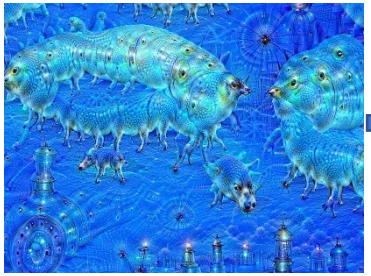
0.12 black widow
0.04 cockroach
0.01 cat
0.05 golfcart
0.02 dog
0.12 mite
0.81 moon jelly
…

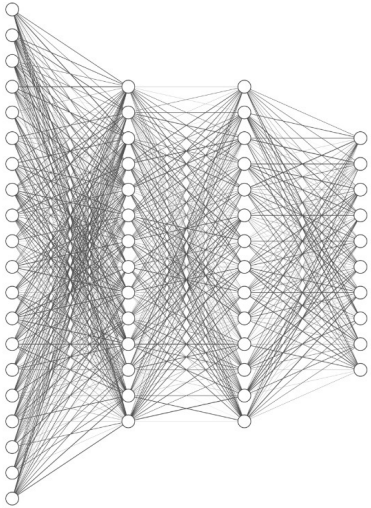
$$L = \sum_{j=0}^{neurons} \left(a_j^{(L)} - d_j \right)^2$$



0.11 black widow
0.08 cockroach
0.91 cat
0.05 golfcart
0.01 dog
0.02 mite
0.01 moon jelly
…

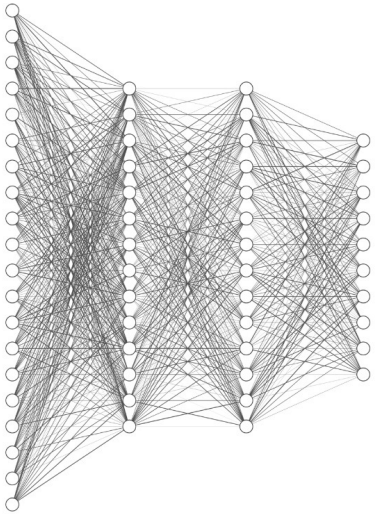
$$L = \sum_{j=0}^{neurons} \left(a_j^{(L)} - y_j \right)^2$$





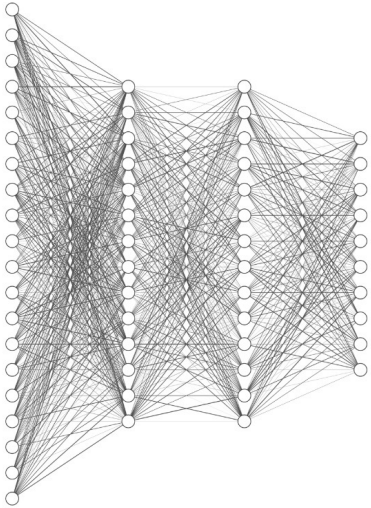
0.11 black widow
0.18 cockroach
0.21 cat
0.15 golfcart
0.21 dog
0.04 mite
0.41 moon jelly
…

$$L = \sum_{j=0}^{neurons} \left(a_j^{(L)} - d_j \right)^2$$



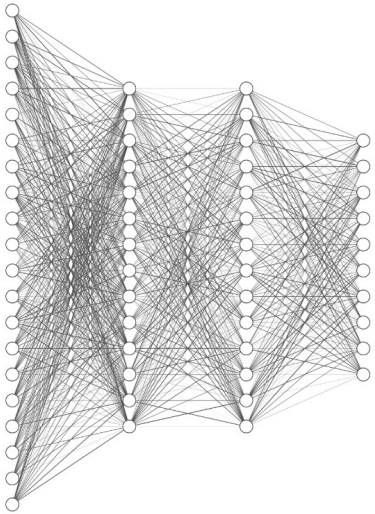
0.11 black widow
0.08 cockroach
0.91 cat
0.05 golfcart
0.01 dog
0.02 mite
0.01 moon jelly
…

$$L = \sum_{j=0}^{neurons} \left(a_j^{(L)} - y_j \right)^2$$



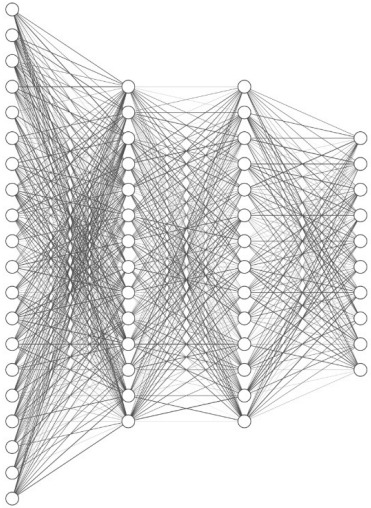
0.04 black widow
0.05 cockroach
0.04 cat
0.02 golfcart
0.81 dog
0.09 mite
0.22 **moon jelly**
…

$$L = \sum_{j=0}^{neurons} \left(a_j^{(L)} - d_j \right)^2$$



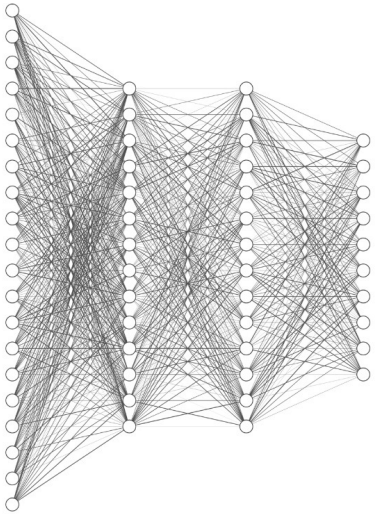
0.11 black widow
0.08 cockroach
0.91 cat
0.05 golfcart
0.01 dog
0.02 mite
0.01 moon jelly
…

$$L = \sum_{j=0}^{neurons} \left(a_j^{(L)} - y_j \right)^2$$



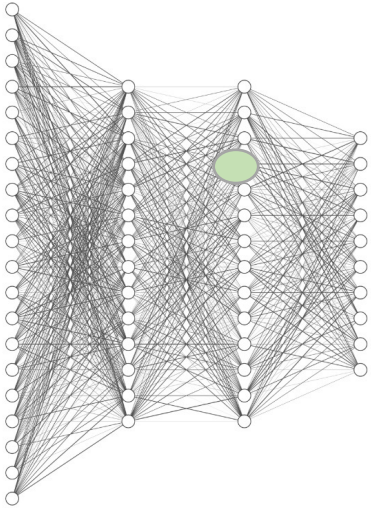
0.04 black widow
0.05 cockroach
0.04 cat
0.02 golfcart
0.81 dog
0.09 mite
0.22 **moon jelly**
…

$$L = \sum_{j=0}^{neurons} \left(a_j^{(L)} - d_j \right)^2$$

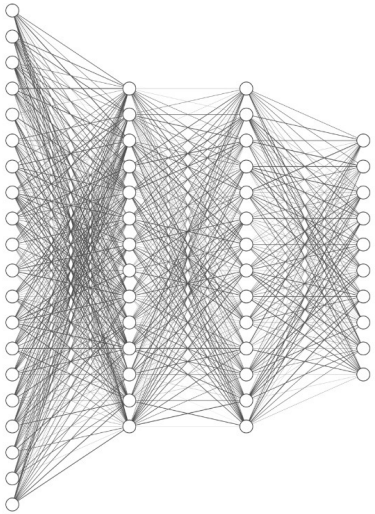


0.11 black widow
0.08 cockroach
0.91 cat
0.05 golfcart
0.01 dog
0.02 mite
0.01 moon jelly
…

$$L = \sum_{j=0}^{neurons} \left(a_j^{(L)} - y_j \right)^2$$

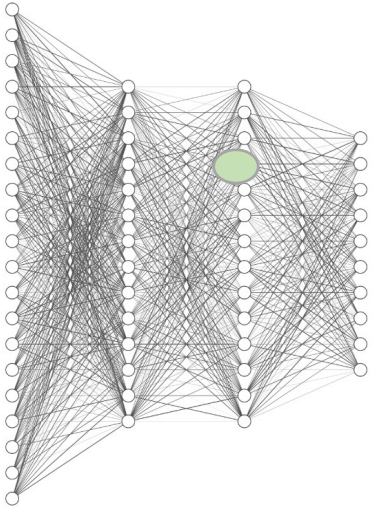


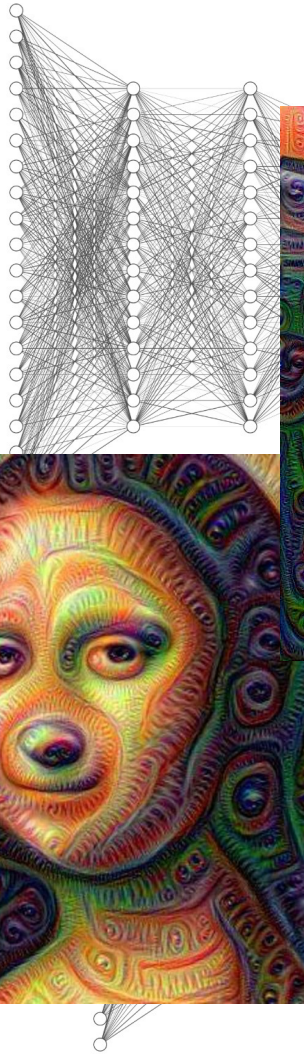
$$L = \sum_{j=0}^{neurons} \left(a_j^{(L)} - d_j \right)^2$$

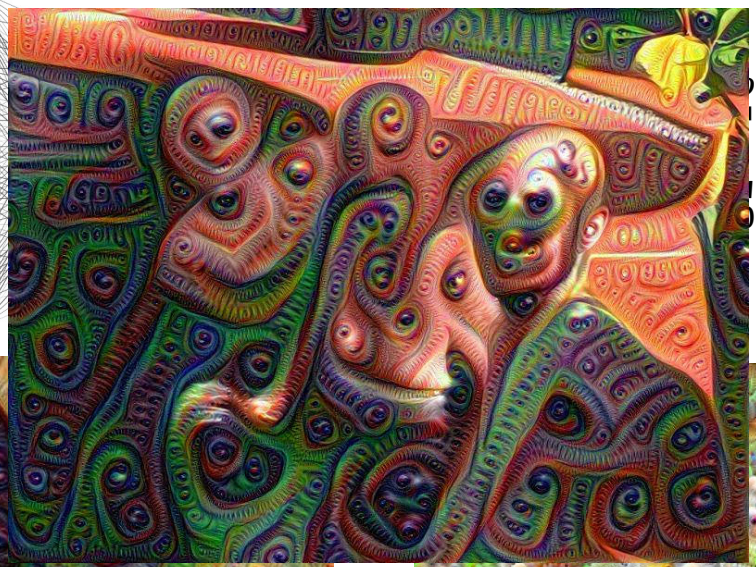


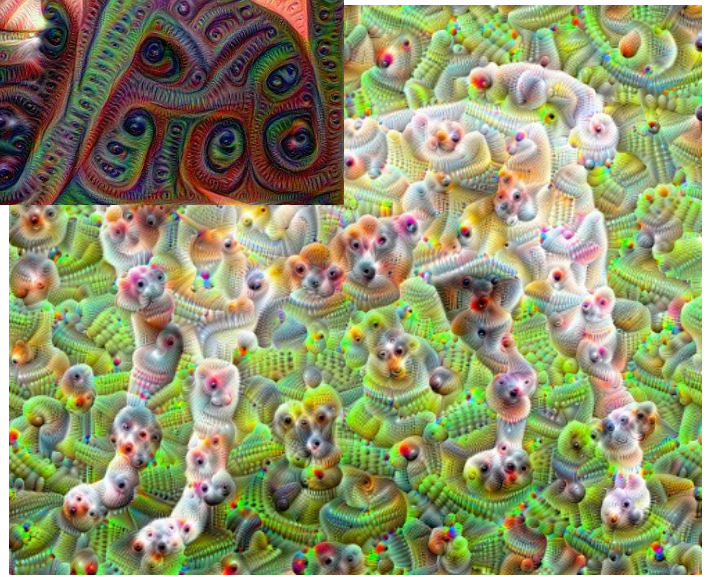
0.11	black widow
0.08	cockroach
0.91	**cat**
0.05	golfcart
0.01	dog
0.02	mite
0.01	moon jelly
…	

$$L = \sum_{j=0}^{neurons} \left(a_j^{(L)} - y_j \right)^2$$









$$\left(a_j^{(L)} - y_j\right)^2$$

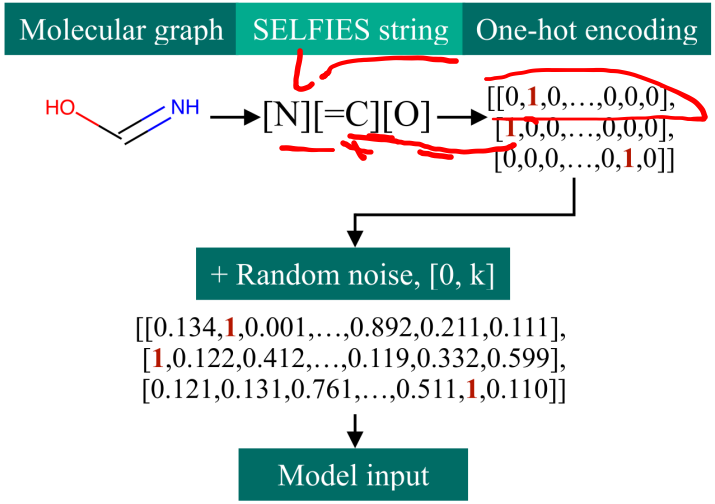
Deep molecular dreaming

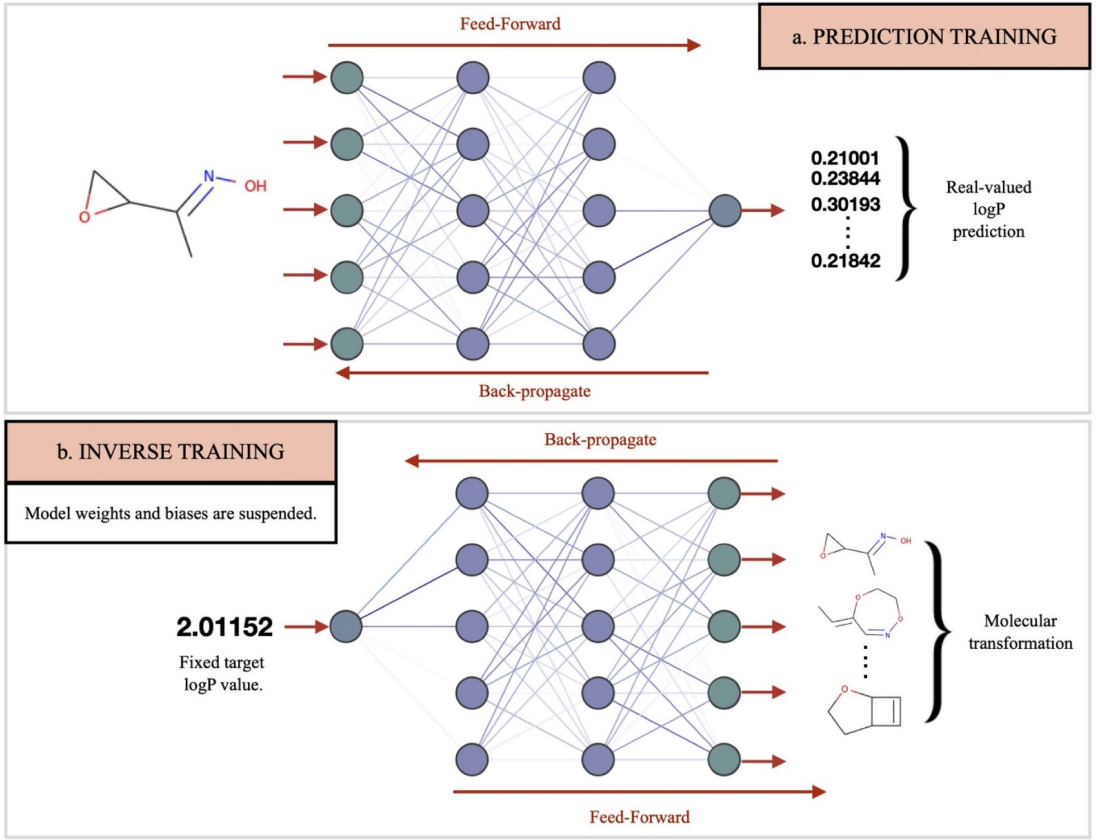
Cynthia Shen *et al* 2021 *Mach. Learn.: Sci. Technol.* **2** 03LT02

Figure 1. Inceptionism techniques aim to understand the inner mechanism of a neural network by visualizing the information encoded in a particular layer [1]. A typical result after training on a chosen image is a build-up of flourishes that has a dream-like quality—hence the name of the process, 'dreaming'. To generate this image, we use the following github repository [2].

Deep molecular dreaming

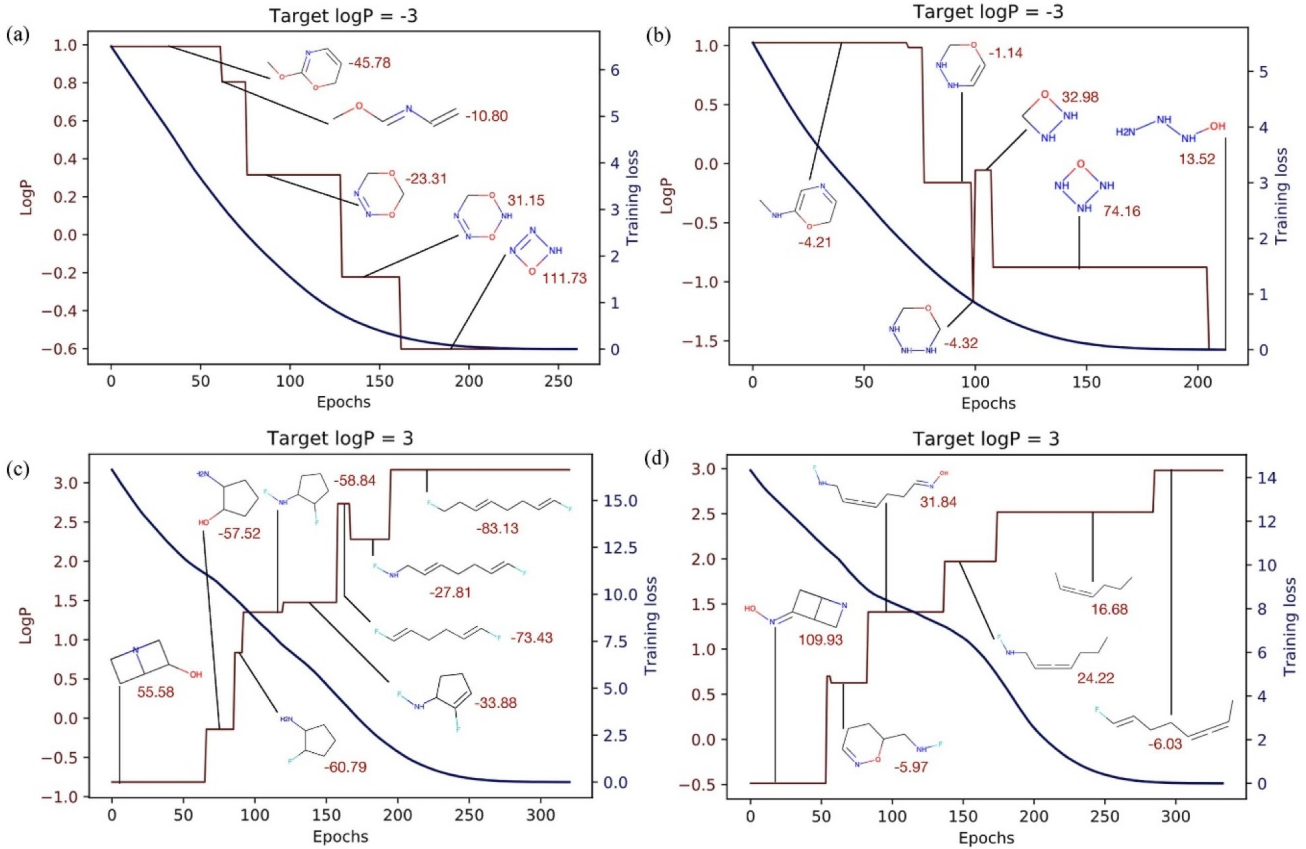
Cynthia Shen *et al* 2021 *Mach. Learn.: Sci. Technol.* **2** 03LT02





Deep molecular dreaming

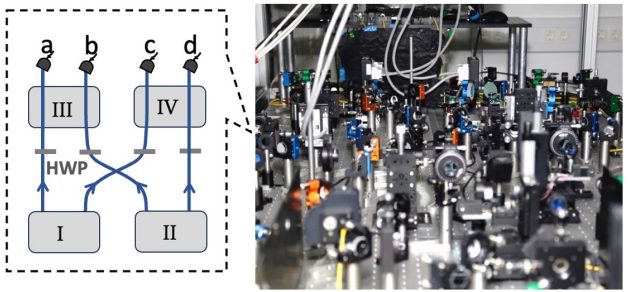
Cynthia Shen *et al* 2021 *Mach. Learn.: Sci. Technol.* **2** 03LT02



Quantum Experiments with Deep Dreaming

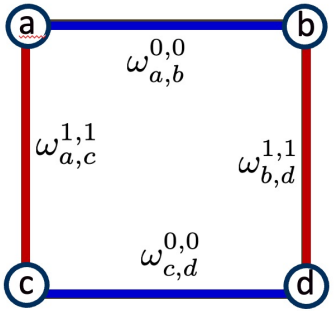
Tareq Jaouni et al 2024 Mach. Learn.: Sci. Technol. 5 015029

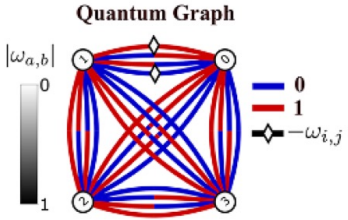
Quantum experiments



Vertex: Photonic path
Edge: Correlated Photon pair
Edge weight: amplitude
Color: Photonic Mode

Graph representation



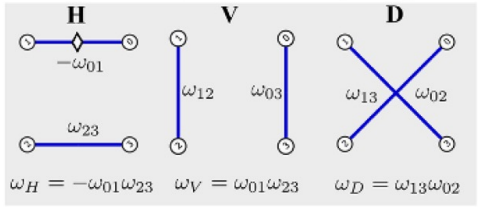


Model Input

$\omega = [\omega_{01}, \omega_{12}, \omega_{23}, ...]$

$\omega_{a,b} \in [-1, 1]$

Perfect Matching Directions e.g. for |0000>



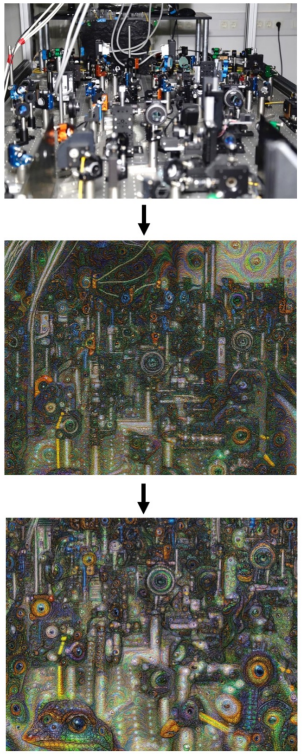
$$|\Phi(\omega)\rangle \propto (\omega_H + \omega_V + \omega_D)\,|0000\rangle$$

Quantum State from Graph

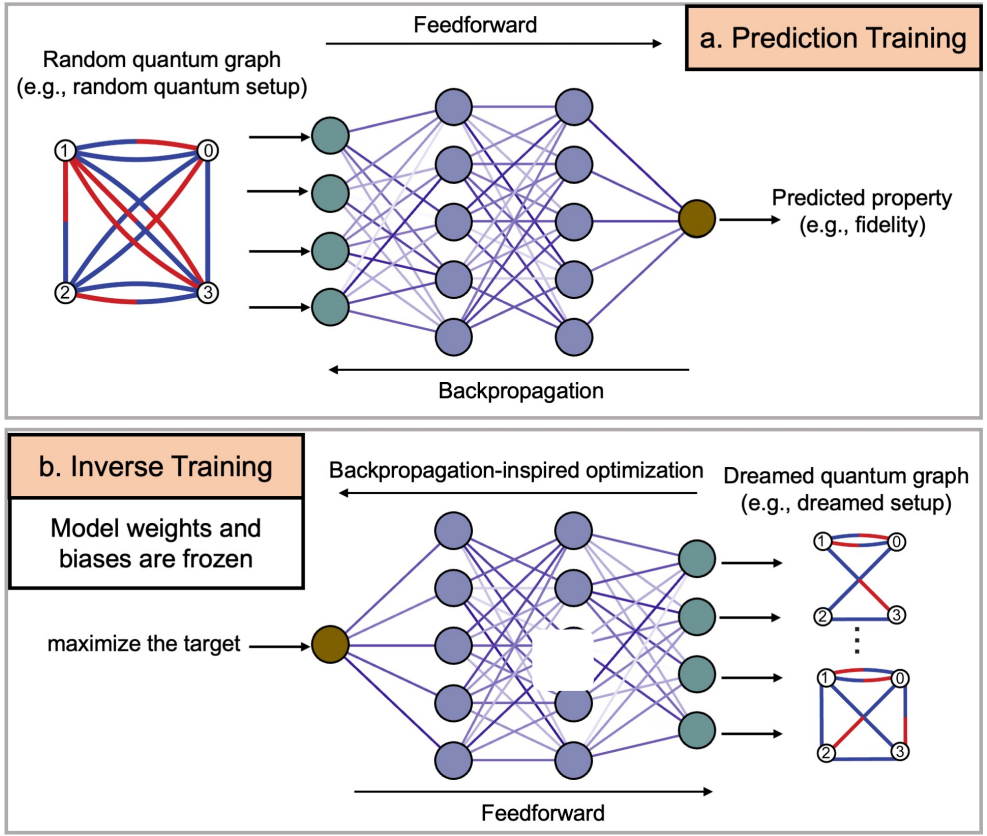
Quantum Experiments with Deep Dreaming

Tareq Jaouni et al 2024 Mach. Learn.: Sci. Technol. 5 015029

DeepDreaming for Images

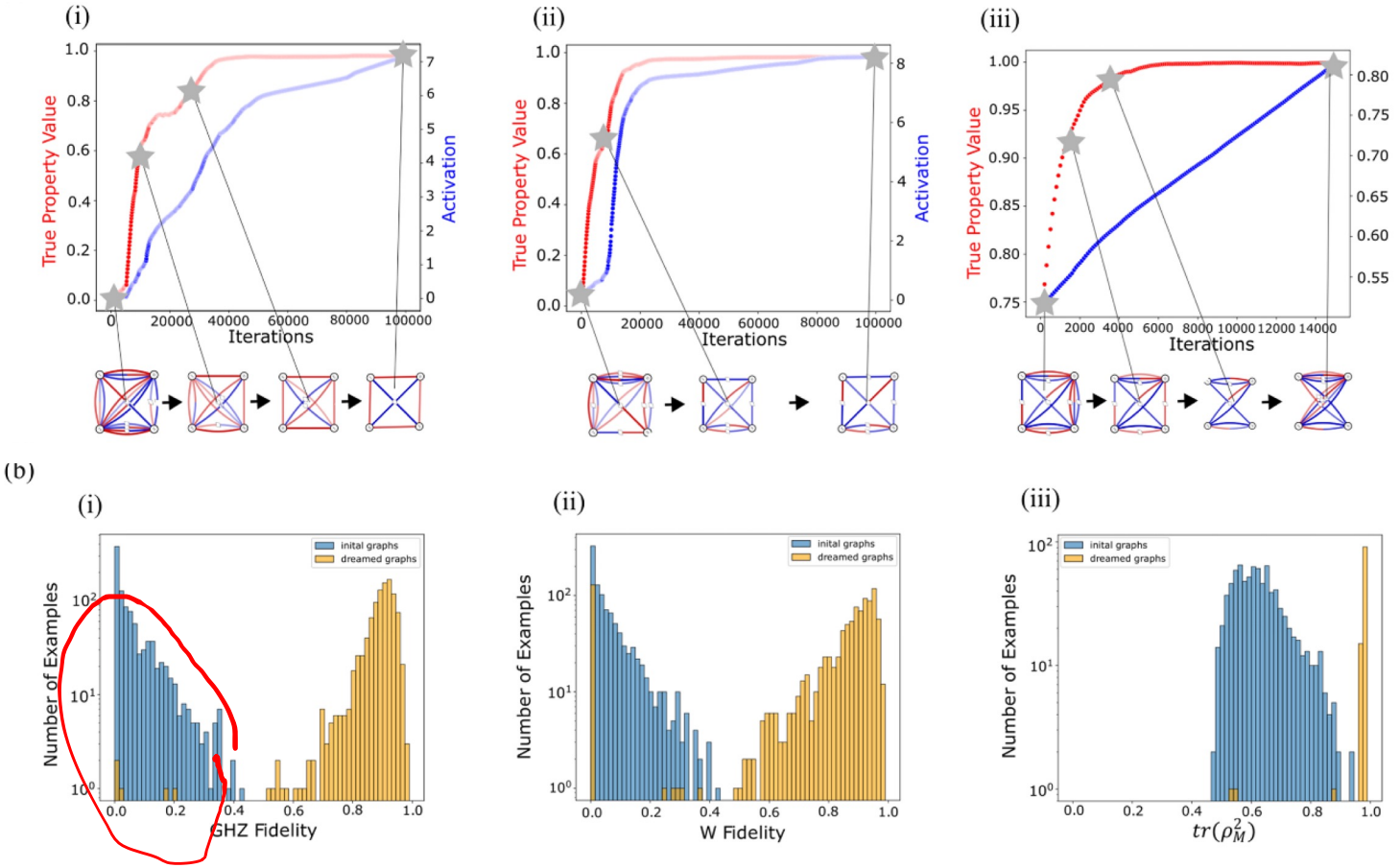


Deep Quantum Setup Dreaming



Quantum Experiments with Deep Dreaming

Tareq Jaouni et al 2024 Mach. Learn.: Sci. Technol. 5 015029



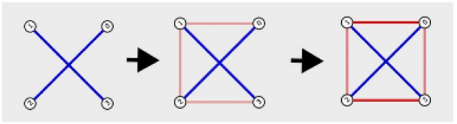
Quantum Experiments with Deep Dreaming

Tareq Jaouni et al 2024 Mach. Learn.: Sci. Technol. 5 015029

Discover quantum graphs maximizing

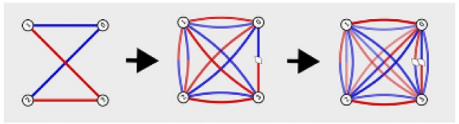
(a) $|\langle\Phi(\omega)|\mathrm{GHZ}\rangle|^2$

$|\Phi(\omega)\rangle_{in} = |0000\rangle$



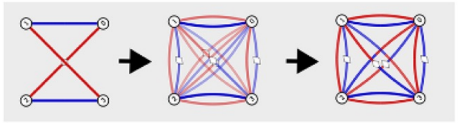
Extracted Strategy Generations of disjoint red-cycle graph

$|\Phi(\omega)\rangle_{in} = |0011\rangle + |0101\rangle$



Extracted Strategy Completion of |0000> and |1111> PMs in the H and D directions, Mitigation of unwanted terms via negative PMs in the unused direction

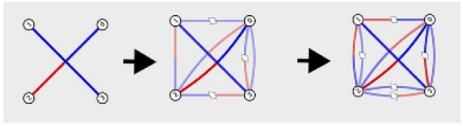
$|\Phi(\omega)\rangle_{in} = |0000\rangle - |1111\rangle$



Extracted Strategy Generation of positive-valued, disjoint red-cycle graph, reduction of edge weights of the -|1111> D PM

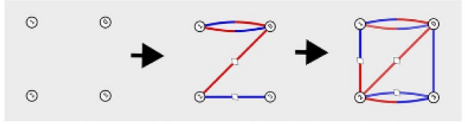
(b) $|\langle\Phi(\omega)|\mathrm{W}\rangle|^2$

$|\Phi(\omega)\rangle_{in} = |0010\rangle$



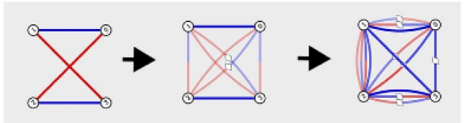
Generation of |0001> and |0100> PMs in unused directions, completion of |1000> D PM

$|\Phi(\omega)\rangle_{in} = 0$



Creation of W-State PMs in H and V directions, compenstation of consequent |0101> term via negative disjoint PM

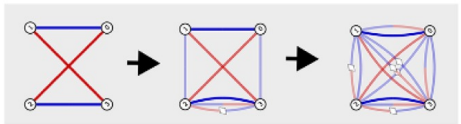
$|\Phi(\omega)\rangle_{in} = |\mathrm{GHZ}\rangle$



Reduction of edge weights of |1111> D PM, Completion of blue-cycle graph, which assists the generation of W-state PMs

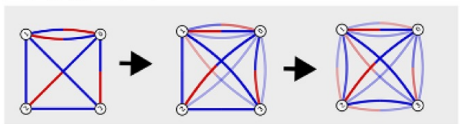
(c) Mean of $\mathrm{tr}(\rho^2_{\mathcal{M}})$ across all bipartitions $\mathcal{M}$

$|\Phi(\omega)\rangle_{in} = |\mathrm{GHZ}\rangle$



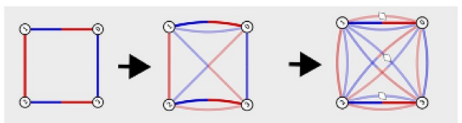
Reduction of edge weights for |1111> D PM, Generation of PMs in all directions terms seperable to |0000> across atleast two bipartitions

$|\Phi(\omega)\rangle_{in} = |\mathrm{W}\rangle$



Reduction of edge weights for |1000> V PM, Generation of red-blue cycle graph to create |0101> PMs

$|\Phi(\omega)\rangle_{in} = |0110\rangle + |1001\rangle$



Mitigation of |0110> term via disjoint negative PM and edge weight reduction of the V PM, Generation of terms seperable to |1001>